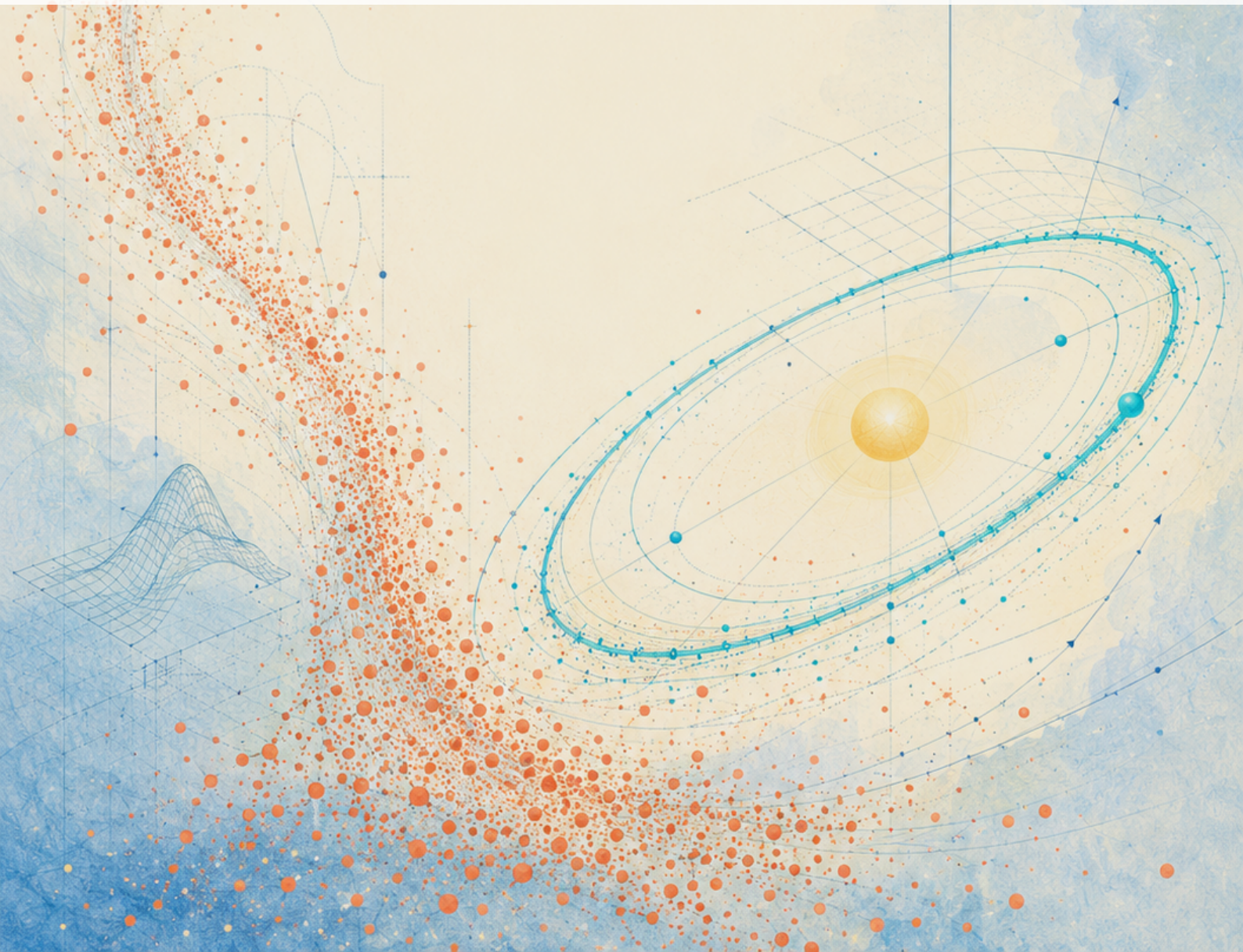
חישוב מדעי לפיסיקאים

תכנות בפיסיקה עם Julia ו-Python

77641



תוכן העניינים

-1 על הקורס ועל חומרי הלימוד . . . 1

-1.1 מפת השנה . . . 1

-1.2 החומרים שילוו אותנו בכל שבוע . . . 1

-1.3 איך לקרוא את הספר . . . 1

-1.4 למה מספר הפרק הוא -1? . . . 2

0 מבוא: פיסיקה בעידן החישובי . . . 3

0.1 למה להשתמש במחשבים כדי לעשות פיסיקה? . . . 3

0.1.1 נתחיל ממדידות ותנועה . . . 3

0.1.2 כוכב שנקרע בידי חור שחור . . . 3

0.1.3 בעקבות מועמד היגס ב־CERN . . . 4

0.2 מהי תוכנית? מהו מחשב? . . . 5

0.3 למה להתחיל ב-Julia? . . . 5

0.4 למה Python? . . . 7

0.4.1 דוגמה מלאה: האות בשני אתרי LIGO . . . 7

0.5 למה ללמוד את שתיהן? . . . 9

0.6 כיצד שנת הלימודים בנויה? . . . 9

0.7 מה נלמד? . . . 9

0.8 טיזר: לשחזר את ניתוח מאדים של קפלר . . . 10

0.8.1 קבלת מאגר הקורס . . . 10

0.9 סביבת הקורס: התקנה חד־פעמית והרצה ניתנת לשחזור . . . 11

1 חישובים ראשונים ב-Julia והתאמה מודרכת לחוק אוהם . . . 13

1.1 היכן רואים תוצאה? . . . 13

1.2 Julia כמחשבון . . . 13

1.3 שמות לערכים לפני הפיזיקה . . . 14

1.4 תהליך מודרך ממדידה למודל . . . 14

1.4.1 לבנות את הניתוח, צעד גלוי אחר צעד . . . 15

1.5 תרגול: מדידת קבוע קפיץ . . . 21

1.6 מטלת החובה: הסקת תאוצת הכובד ממדידות נפילה חופשית . . . 21

1.7 בונוס לבחירה: גרף שמעביר תוצאה מדעית . . . 21

1.8 להעמקה: עקומות ליסאז'ו . . . 21

2 ערכים, החלטות וגוף על מישור משופע . . . 23

2.1 הרצאה: ממשתנים לאלגוריתם . . . 23

2.1.1 משתנה נותן שם לערך . . . 23

2.1.2 התוכנית מתבצעת לפי הסדר . . . 24

2.1.3 אפשר להציב במשתנה ערך חדש . . . 24

2.1.4 לערכים יש טיפוסים . . . 24

2.1.5 ארבעה טיפוסי ערכים פשוטים . . . 25
2.1.6 השוואות יוצרות ערכי אמת ושקר . . . 25
2.1.7 החלטה ראשונה בעזרת if . . . 26
2.2 קינמטיקה בחלקים כהחלטה . . . 26
2.2.1 כמה ענפים . . . 27
2.3 לערכים יש טיפוסים . . . 28
2.4 בוליאנים והחלטות הדרגתיות . . . 28
2.5 זוויות, לוגיקה ואימות לפני חישוב . . . 29
2.6 מודל חיכוך בחלקים על מישור משופע . . . 30
2.7 תרגול: משיכה יכולה להפוך את כיוון התנועה הצפוי . . . 31
2.8 מטלת החובה: האם קליע עובר מעל קיר? . . . 31
2.9 בונוס לבחירה: יחידות ופאזורים בזרם חילופין . . . 32
2.10 להעמקה: החזרה פנימית מלאה . . . 32

3 לולאות, וקטורים ותנועה נדגמת . . . 33
3.1 וקטורים שומרים היסטוריה מסודרת . . . 33
3.1.1 יצירת וקטורים . . . 33
3.1.2 אינדוקס ופרוסות . . . 33
3.1.3 שינוי וקטור . . . 34
3.1.4 חשבון וקטורים ופונקציות שימושיות . . . 35
3.2 חזרה מתוכננת: לולאת for . . . 36
3.3 סיכומים: ממוצע וסטיית תקן מדגמית . . . 39
3.4 חזרה המותנית בתנאי: while . . . 39
3.5 מהנגזרת לעדכון אוילר . . . 40
3.6 עידון: השוואה לתשובה ידועה . . . 42
3.7 תרגול: זמן קירור נדגם ראשון . . . 43
3.8 תרגיל בית ראשי: זריקת פטיש עם התנגדות אוויר . . . 43
3.9 בונוס אופציונלי: שלוש סכמות אינטגרציה . . . 43
3.10 המשך חקר: מהליכה אחת לדיפוזיה . . . 43

4 מטריצות, לולאות וסיבובים . . . 45
4.1 כתיבת מטריצה וצורתה . . . 45
4.1.1 אינדוקס מטריצה . . . 45
4.2 לולאות while: חזרה עד שתנאי נעשה שקרי . . . 46
4.2.1 קלט עם ערך סיום: ממוצע הכיתה . . . 46
4.2.2 בדיקת תקינות הקלט . . . 47
4.2.3 משחק ניחושים . . . 47
4.2.4 מדידת זמן של לולאת while: הטור ההרמוני . . . 48
4.3 מכפלת מטריצה בווקטור ובמטריצה . . . 49
4.4 לולאות מקוננות: לוח כפל . . . 49
4.5 לולאות מקוננות: כפל מטריצות . . . 50
4.6 בדיקה אם מטריצה סימטרית . . . 51
4.7 מטריצות סיבוב בשני ממדים . . . 52
4.7.1 בדיקה שמטריצה אכן מתארת סיבוב . . . 52
4.8 סיבוב מלבן במישור . . . 53

5 פונקציות בפיסיקה . . . 55
5.1 מחישוב לפונקציה . . . 55
5.2 תחום הראות: היכן שם מוגדר . . . 56
5.3 ארגומנטים מיקומיים, ארגומנטי מפתח וערכי ברירת מחדל . . . 57

5.4 החזרת ערך 58
5.5 שינוי מערכים ומוסכמת ! 58
5.6 הפעלה על איברי מערך 59
5.7 הבנות רשימה ומטריצה 60
5.8 מחלקים ומספרים ראשוניים 60
5.9 אינטגרציית אוילר למטוטלת 61
5.10 המחזור כתלות בזווית השחרור 62
5.11 מבט קדימה 62



1-. על הקורס ועל חומרי הלימוד

למה ללמוד תכנות ולהשתמש במחשבים בפיסיקה? הם מאפשרים לנו להפוך מודל לחישוב שאפשר לחזור עליו, לשנות אותו ולהשוות את תוצאותיו למדידות. לאורך הקורס נשתמש בחישוב כדי לחבר בין רעיונות מפיסיקה וממתמטיקה, נתונים, שיטות נומריות והשאלות המדעיות שנרצה לחקור.

1.1- מפת השנה

נתחיל ב־Julia ובחישובים קטנים וברורים שיעזרו לנו לבנות ביטחון: קריאת טבלה שסופקה לנו, הצגת מדידות בגרף, התאמה של קו ישר ומעקב אחר תנועה פשוטה. בהמשך נכתוב בעצמנו אלגוריתמים לשיטות נומריות, נשווה אותם לחבילות קיימות ונשלב אותם בחקירות רחבות יותר. בהמשך נשתמש ב־Python כדי להעביר רעיונות מדעיים שכבר הכרנו לשפה נוספת ולאקוסיסטם המחקרי הרחב שלה. פרק 0 מציג את הנושא; מפרק 1 ואילך, מספר הפרק זהה למספר השבוע בקורס.

1.2- החומרים שילוו אותנו בכל שבוע

לחומרים של כל שבוע יש תפקיד משלהם:

- **ספר ההרצאות והמחברת:** הספר מפתח את הרעיונות בהדרגה, עם דוגמאות פתורות והפלט שלהן. מחברת הרצאה מלאה הולכת באותו מסלול בפורמט שאפשר להריץ, כך שנוכל לבדוק את הדוגמאות ולחקור אותן. פתיחת מחברת להרצה היא רשות, ואפשר לעשות זאת ב-VS Code עם הרחבת Jupyter או ב-JupyterLab; המחברות כוללות גם פלט שמור. מצגות Reveal תומכות בהרצאה בכיתה.
- **מדריך התרגול והמחברת:** נתחיל בחזרה קצרה ובדוגמה פתורה אחת, ואז נקדיש את רוב התרגול לעבודה על תרגיל הליבה בעזרת צוות ההוראה. המטלה עצמה נשארת מסמך נפרד.
- **תרגיל ליבה:** המטלה השבועית נותנת לנו הזדמנות לתרגל את החומר, ופתרון מצורף תומך בהוראה ובחזרה. אנחנו מגישים אותה בעמוד ה-Moodle של הקורס, שם נמצאים המטלה וקישור ההגשה. המטלה הראשונה נבדקת ידנית בשבוע 1: מעלים סקריפט ואיור. מהשבוע השני, שאלות תכנות מתאימות ב־CodeRunner נבדקות אוטומטית באמצעות הרצת בדיקות גלויות על ההגשה; בשאלה מוסבר מה להגיש ומה הבדיקות בודקות.
- **לימוד רשות:** כל בונוס שהושלם מזכה בשליש נקודה לציון הסופי. הבונוס הוא רשות, אינו נכלל בבחינה ואינו תנאי לחומר של שבוע מאוחר יותר. חלק נפרד בשם להעמקה מציע למידה עצמית מעבר לליבת הקורס, ושאלות תרגול שבועיות מאפשרות לנו לבדוק את ההבנה לקראת הבחינה.

1.3- איך לקרוא את הספר

תיבות צבעוניות מאפשרות לנו לעצור לרגע ולהעמיק ברעיון שראוי להסבר משלו:

- **תיבות מחשוב באפור־כחול** מסבירות כלי תכנות, מוסכמה או דרך עבודה.
- **תיבות פיסיקה בסגול** מציגות את המודל הפיסיקלי, את הגדלים שבו ואת גבולותיו.
- **תיבות מתמטיקה בירוק** מפתחות קשר מתמטי או גזירה.
- **תיבות בכתום** מציעות רמז, אזהרה או נקודה שכדאי לזכור.

קטעי קוד מופיעים בתיבות תכלת בהיר. התיבה הירקרקה שמתחת לדוגמת קוד מציגה את הפלט שלה. נשווה בין הפלט להסבר שאחריו, ונאמץ את אותה דרך עבודה כשנריץ מחברת בעצמנו.

-1.4 למה מספר הפרק הוא -1?

המדריך הזה מופיע לפני הפרק הממוספר הראשון, ולכן מספרו -1. פרק 0 הוא המבוא לתכנות בפיסיקה. שבוע 1 הוא פרק 1, שבוע 2 הוא פרק 2, וכך המספור ממשיך לאורך השנה. כך, כשנדבר על שבוע 6 או על פרק 6, נדע שאנחנו מתכוונים לאותו חלק בקורס.



0. מבוא: פיסיקה בעידן החישובי

0.1 למה להשתמש במחשבים כדי לעשות פיסיקה?

למה אנחנו צריכים תכנות ומחשבים בפיסיקה? הם עוזרים לנו לעקוב אחרי רעיון ממודל לחישוב, ומחישוב למדידות שאפשר לבחון. אפשר להתחיל בשאלות מוכרות ואז לראות כיצד אותה גישה מגיעה לבעיות בחזית המחקר.

0.1.1 נתחיל ממדידות ותנועה

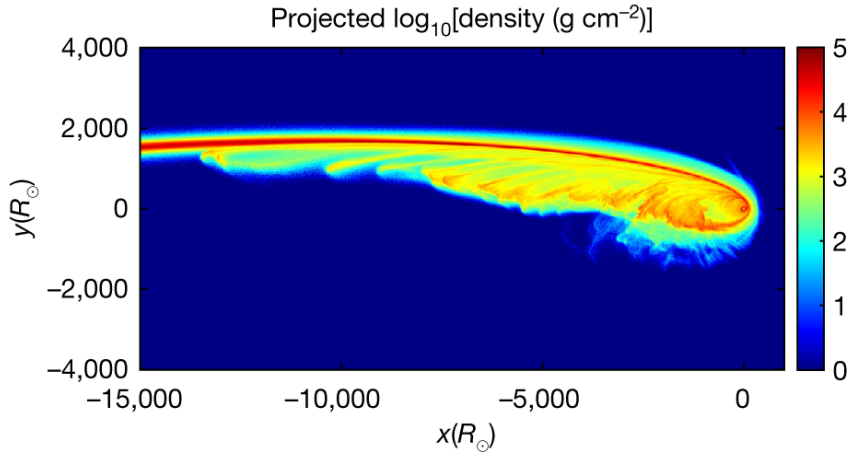
נניח שאנחנו מודדים מתח וזרם במוליך. גרף מאפשר לנו לראות כיצד שני הגדלים משתנים יחד; התאמת ישר נותנת לנו אומדן להתנגדות כאשר המוליך מתנהג בצורה אוהמית. בשבוע 1 נעבוד עם דוגמה מסופקת של מדידה–גרף–התאמה. בהמשך נבנה את שיטת ההתאמה בעצמנו ונבחן כיצד השאריות ואי־הוודאות במדידה משפיעות על המסקנה.

במכניקה, מודל של קליע אידיאלי מוביל למסלול פרבולי שאפשר לחשב בעזרת נוסחאות מוכרות. מה משתנה כשמוסיפים התנגדות אוויר שתלויה במהירות, או צפיפות אוויר שמשתנה עם הגובה? הוספת ההשפעות האלה קושרת בין המיקום למהירות במשוואות התנועה. אינטגרציה נומרית מקדמת אותן בצעדי זמן קטנים, ומאפשרת להשוות בין המסלול האידיאלי לתחזית מפורטת יותר.

0.1.2 כוכב שנקרע בידי חור שחור

מה קורה כשכוכב חולף קרוב מדי לחור שחור מסיבי? המשיכה הכבידתית של החור השחור חזקה יותר בצד הקרוב לכוכב מאשר בצדו הרחוק. כשההבדל הזה גובר על הכבידה העצמית של הכוכב, הוא נקרע. הגז נמתח לזרמים, חוזר ומתנגש, מתחמם ומתחיל ליצור זרימת ספיחה.

הערכות אנליטיות מספרות לנו היכן מתחיל השיבוש וקובעות סקלות זמן אופייניות למסלול. כדי לעקוב אחר הזרמים החוזרים כשהם מתנגשים, יוצרים גלי הלם ופולטים קרינה, חוקרים פותרים נומרית משוואות הידרודינמיות ומשוואות קרינה מצומדות לאורך מרחב תלת־ממדי וזמן. האיור שלמטה מציג את צפיפות העמודה המוקרנת שהתקבלה מחישוב כזה, שבוצע בידי שניים מחברי המחלקה שלנו; סקלת הצבעים שלו עוקבת אחר המקום שבו הגז המשובש מרוכז. סימולציות כאלה גם חוזות כיצד משתנה אורו של האירוע ככל שהזרימה מתפתחת, ומאפשרות תחזיות שאינן אפשריות כלל בחישובים אנליטיים.

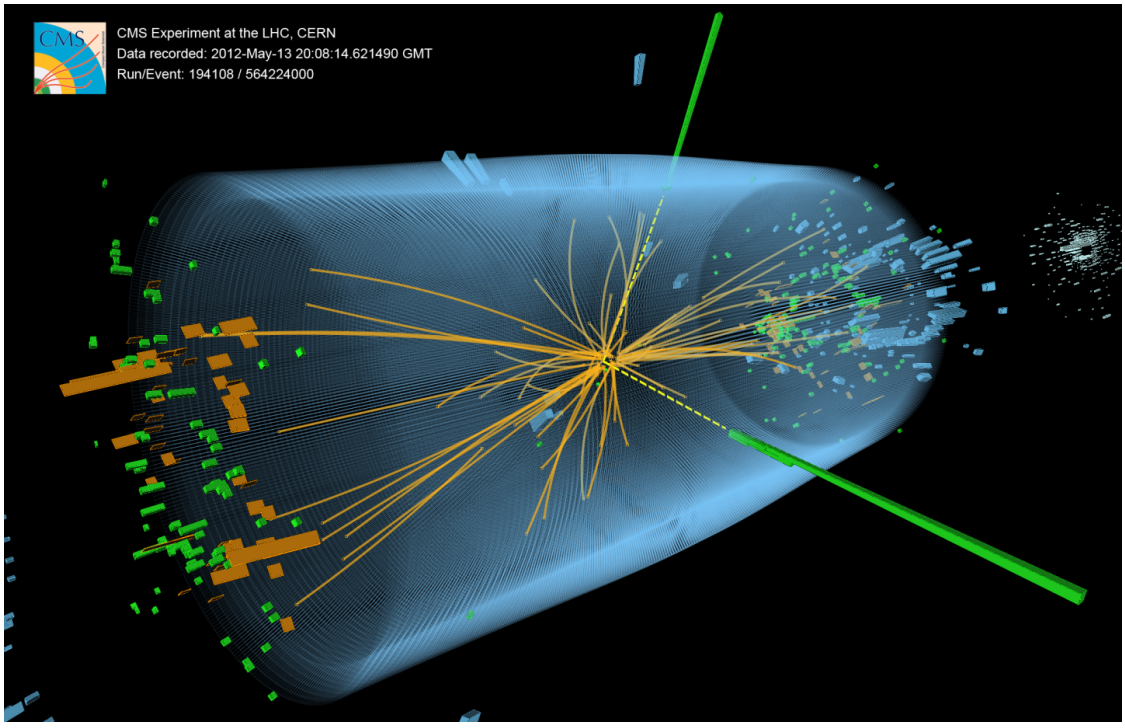


לוגריתם צפיפות העמודה המוקרנת בסימולציה של אירוע שיבוש גאותי; המיקומים נמדדים ברדיוסים של השמש. מתוך E. Steinberg and N. C. Stone, "Stream–disk shocks as the origins of peak light in tidal disruption events," Nature 625, 463–467 (2024), Fig. 1, doi:10.1038/s41586-023-06875-y.

0.1.3 בעקבות מועמד היגס ב־CERN

ב־CERN בוזון היגס נוצר בהתנגשויות חלקיקים נדירות מאוד ומתפרק כמעט מיד. הגלאים ב־CERN רושמים מיליוני אירועי התנגשות. בכל אחד מהם יש לחשב את הדינמיקה של כל החלקיקים המעורבים מתוך האנרגיות והכיוונים הנמדדים של החלקיקים הנוצרים. זוהי משימה בלתי אפשרית לחלוטין לבני אדם רגילים, ולכן ניתוח ממוחשב הוא הכרחי.

האיור שלמטה מציג אירוע מסוים שיצר שני פוטונים, והוא מועמד סביר מאוד להיווצרותו ולהתפרקותו של חלקיק ההיגס. חלקיק זה הונח תאורטית כדי להסביר כיצד חלקיקים מסוימים קיבלו את המסה שלהם.



מועמד דו־פוטוני של *CMS* מחיפוש ההיגס, המשורטט מתוך אותות גלאי משוחזרים. תמונה: CMS/CERN, סקירת CERN על בוזון היגס.

שתי הדוגמאות האלה מראות לנו כיצד אפשר להשתמש במחשבים כדי לנתח נתונים ולמדל מערכות בדרכים שהן שונות איכותית וכמותית, ומציעות לנו, הפיסיקאים, עוד כלים.

0.2 מהי תוכנית? מהו מחשב?

תוכנית היא רצף הוראות שמסביר למחשב כיצד לבצע משימה. בדוגמת הקליע נוכל להתחיל ממקום ומהירות, לחשב את התאוצה ולעדכן את התנועה בזמנים עוקבים. כדי לכתוב את ההוראות נצטרך לציין אילו נתונים לשמור, כיצד לחשב בעזרתם ומתי לחזור על פעולה או לעצור. קורא אנושי יכול לעיתים להשלים צעד חסר בגזירה; למחשב נצטרך לכתוב גם אותו במפורש.

מחשב הוא מכונה פיסיקלית שמבצעת את ההוראות האלה שוב ושוב. המעבד המרכזי (CPU) מבצע פעולות אריתמטיות ולוגיות; הזיכרון (RAM) מחזיק את הנתונים וההוראות שנמצאים בשימוש כעת; אחסון קבוע שומר קבצים לאחר שהתוכנית מסתיימת; והתקני קלט/פלט מחברים את המכונה לקבצים, מסכים, מכשירים ורשתות. כאשר נריץ סקריפט ב-Julia, השפה תהדר את הוראות המקור שכתבנו להוראות מכונה שהמעבד יכול לבצע; מערכת ההפעלה מנהלת קבצים, זיכרון וגישה למסך.

כשנקרא תוכנית, נוכל לעקוב אחר הנתונים מערכי ההתחלה, דרך החישובים ועד לתוצאה. אלגוריתם מתאר את צעדי החישוב, והמודל הפיסיקלי עוזר לנו לפרש את התוצאה. נבנה את דרך העבודה הזאת בהדרגה: בשבוע 2 נכיר טיפוסים, המרת קלט והחלטות; בשבוע 3 נכיר לולאות ותנאי עצירה; ובשבוע 5 נלמד להגדיר פונקציות.

0.3 למה להתחיל ב-Julia?

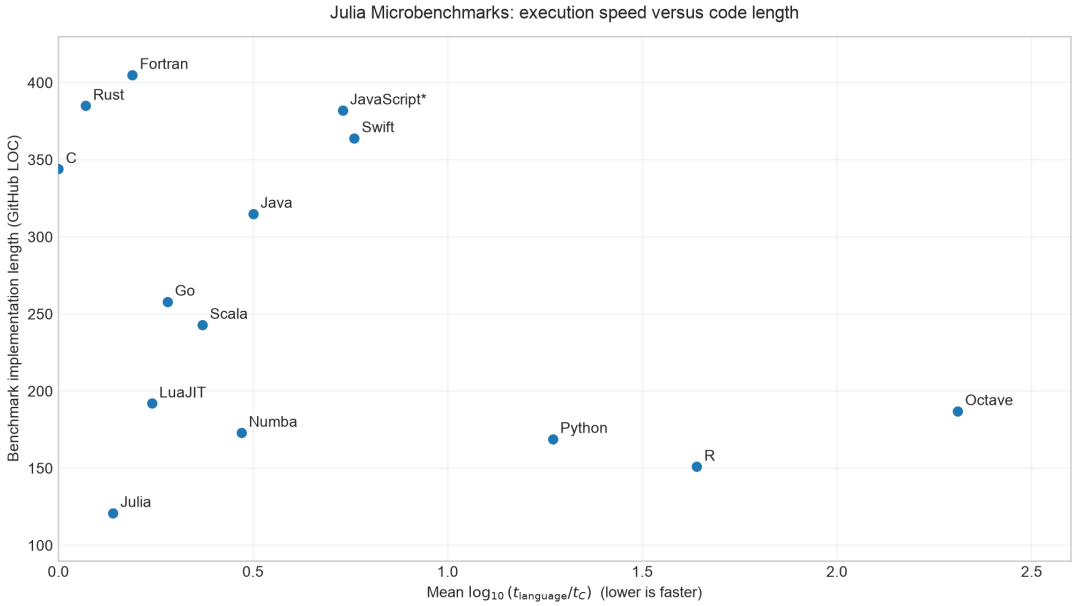
Julia נבנתה לעבודה נומרית ומדעית. יש לה תחביר ישיר עבור וקטורים, מטריצות, אלגברה לינארית, שיגור מרובה ופותרי משוואות דיפרנציאליות, והיא מתאימה גם לסקריפטים ולמחברות חקירה. היא גם מהירה מספיק כך שלעיתים קרובות הגרסה הקריאה הראשונה של מודל נומרי כבר שימושית.

עבורנו כפיסיקאים, היתרון הראשון והחשוב הוא שהקוד יכול להיראות כמו המודל. וקטור של מדידות, מטריצה שמייצגת העתקה לינארית ומערך של ערכים שנדגמו בזמן הם אובייקטים רגילים ב-Julia. ההבחנה בין ערך יחיד, וקטור וחישוב איבר-איבר נראית בתחביר. כך קל יותר לבדוק ממדים, אינדקסים, יחידות והפעולה הפיסיקלית שהתכוונו אליה, לפני שטעות נומרית נקברת בתוכנית ארוכה.

Julia מאפשרת לנו לכתוב פעולה קרוב לצורה המתמטית שלה, ולחבר פעולות וכלים מוגדרים היטב לחישוב גדול יותר. למשל, אפשר להעביר תוצאה של פעולת מטריצה לחישוב הבא; ואותו רעיון נומרי יכול לפעול על מספרים, וקטורים, מטריצות ובהמשך גם על ערכים שנושאים יחידות. Julia בוחרת את המימוש המתאים לפי טיפוסי הקלט; הרעיון הזה נקרא multiple dispatch. בתחילת הקורס נשתמש בכלים המדעיים, ובהמשך נחזור לרעיון התכנוני הזה כשיעזור להבין כיצד פעולה אחת משתלבת בכמה הקשרים פיסיקליים.

פרויקטים ב-Julia יכולים לרשום בקובצי פרויקט את החבילות ואת הגרסאות ששימשו לניתוח. כך אפשר לחזור לחישוב כעבור חודשים או לשתף אותו עם משתף פעולה, בלי להחליף בשקט את כלי החישוב שמתחתיו. לשפה יש גם ספרייה בסיסית חזקה לאלגברה לינארית נומרית ואקוסיסטם מתרחב למשוואות דיפרנציאליות, אופטימיזציה, סטטיסטיקה, יחידות, גרפים, למידת מכונה ונתונים מדעיים. תחילה נבנה בעצמנו את השיטות המרכזיות, ובהמשך נשתמש בחבילות כמימושים שנבדקו היטב של רעיונות שכבר הבנו.

מהירות חשובה בפיסיקה משום שהיא מאפשרת לבדוק מודל, לבחון תוצאה, לשנות הנחה ולחזור על החישוב. האיור הבא מסכם את חבילת מבחני הביצועים הזעירים של פרויקט Julia. הציר האופקי משווה את זמן הריצה ל-C בסקלה לוגריתמית, ולכן נקודות שמאלה יותר מהירות יותר; הציר האנכי הוא אורך המימוש של המבחן. במבחנים נומריים קטנים אלה Julia קרובה לשפות מהודרות, בעוד קוד הדוגמה נשאר תמציתי. תזמונים אלה מתארים את המבחנים הנבחרים בלבד; מהירותה של תוכנית שלמה תלויה גם באלגוריתם שלה, בארגון הנתונים, בהקצאת הזיכרון ובאופן שבו אנחנו מודדים אותה. הנתונים והתוכניות זמינים ב-https://github.com/JuliaLang/Microbenchmarks.



חשוב במיוחד לקורס זה שהקוד ב-Julia יכול לעקוב מקרוב אחר המבנה המתמטי. נניח שמודל דינמי לינארי נכתב כך:

$$\mathbf{x}_{n+1} = A\mathbf{x}_n + B\mathbf{u}_n, \qquad \mathbf{y}_n = C\mathbf{x}_n.$$

המימוש ב-Julia כמעט זהה:

```
x_next = A * x + B * u
y = C * x
```

כאן x, u ו-y הם וקטורים, ואילו A, B ו-C הן מטריצות. נוכל להשוות כל שורת קוד ישירות למשוואה שלה. בדיקת ממדי המטריצות והיחידות הפיסיקליות תעזור לנו לראות אם החישוב מבטא את המודל שהתכוונו אליו.

יש גם פרטים מעשיים שנכיר במהלך העבודה. ההרצה הראשונה של קוד חדש ב-Julia עלולה להימשך יותר זמן בזמן שהשפה מהדרת קוד מכונה מותאם; הרצות נוספות עשויות להיות מהירות הרבה יותר. הקהילה שלה קטנה מזו של Python, ולכן החבילות הזמינות יכולות להשפיע על בחירת הכלי. נלמד למדוד ביצועים ונשתמש ב-Python כאשר האקוסיסטם שלה מתאים למשימה.

ב-Julia אפשר להקליד ב-REPL וב-VS Code סימנים מתמטיים בעזרת פקודות דמויות LaTeX. למשל, כדי לכתוב ביטוי סקלרי כגון

$$v_1' \cdot (Av_2),$$

מקלידים פקודה שמתחילה בלוכסן הפוך ולוחצים Tab אחרי כל סימן:

- `\alpha<Tab>` יוצר α.
- `\cdot<Tab>` יוצר $\cdot$.
- `v\_1<Tab>\prime<Tab> \cdot<Tab> (A * v\_2<Tab>)` יוצר $\mathrm{v}_1' \cdot (\mathrm{A} * \mathrm{v}_2)$.

מתקבלת הצורה המתמטית שמופיעה למעלה: בשמות משתנים אפשר להשתמש באותיות יווניות, באינדקסים תחתונים, בגרש ובאופרטורים שמופיעים בחישוב.

קריאת המשוואות לצד הקוד

ננסה להתאים שורת קוד לצעד המתאים במודל. זו דרך שימושית למצוא טעות בסימן, אינדקס שנכתב במקום הלא נכון או ממדים שאינם מתאימים, ולבדוק באילו הנחות החישוב משתמש.

0.4 למה Python?

Python היא השפה הדומיננטית באקוסיסטם של נתונים מדעיים. הספריות שלה משמשות למערכים נומריים, גרפים, אופטימיזציה, אסטרונומיה, למידת מכונה, אלגברה סימבולית, עיבוד תמונה ובקרת ניסויים. נלמד Python אחרי שהרעיונות החישוביים המרכזיים כבר יהיו מוכרים, כך שאפשר יהיה לזהות את אותו תהליך עבודה מדעי בשפה שנייה.

Python חשובה במיוחד משום שהיא מחברת ניתוח לסביבת המחקר הרחבה. מעבדה עשויה לספק סקריפט Python למכשיר, משתף פעולה עשוי לשתף מחברת Jupyter, או מאגר ציבורי עשוי לפרסם דוגמה המשתמשת בספריית Python ייעודית. לימוד Python מאפשר לנו לקרוא תהליכי עבודה כאלה, להתאים אותם, להבין מה כל צעד תורם ולבדוק את התוצאה. השפה מועילה גם מעבר לפיסיקה נומרית: טיפול בקבצים, שירותי רשת, אוטומציה, מסדי נתונים והצגה גרפית של נתונים משתמשים באותם כלים בסיסיים.

חלק גדול מן הקוד הנומרי ב-Python מעביר את החישובים הכבדים לספריות מהודרות, כמו NumPy ו-SciPy. כך ביטוי קצר ב-Python יכול לבצע עבודה נומרית רבה ביעילות. לכן זמן הריצה של לולאה שנכתבה ב-Python רגילה עשוי להיות שונה מאוד מזה של חישוב שמבצעת ספרייה נומרית. כשנבחן ביצועים, נבדוק היכן העבודה מתבצעת ונמדוד כמה זמן היא נמשכת.

גם Python קריאה מספיק כדי שניתוח קצר יישמע לעיתים כמו הוראה לעמית. הרשומה שלמטה היא תצוגה מסופקת, ולא שיעור: טרם לימדנו את התחביר של Python, והקוד הזה אינו נבחן. קראו אותו כדי לראות את הצורה של ניתוח שאליו נחזור בהמשך. כאן `dropna` מסיר שורות עם ערכים חסרים בעמודות הנקובות, `2 **` מעלה בריבוע עמודה איבר-איבר, ו-`np.polyfit(..., 1)` מתאים פולינום ממעלה ראשונה ומחזיר את המקדמים שלו, כך ש-`fit[0]` הוא השיפוע:

```
clean = data.dropna(subset=["time_s", "position_m"])
clean["time_squared"] = clean["time_s"] ** 2
fit = np.polyfit(clean["time_squared"], clean["position_m"], 1)
gravity = 2 * fit[0]
```

לשמות יש כאן תפקיד מדעי: הקורא רואה אילו עמודות נבחרו, איזו טרנספורמציה בוצעה, ומדוע השיפוע המותאם קשור לכבידה.

עם המבחר הגדול של חבילות Python, נמצא לפעמים כמה כלים לאותה משימה. נבחר כלים מתועדים היטב, נבין כיצד הם מתאימים למודל שלנו ונרשום את סביבת החבילות כדי שנוכל לחזור לניתוח בהמשך. התאמת דוגמה מספרייה לצרכים שלנו נעשית קלה יותר כשנוכל להסביר מה כל צעד עושה ולבדוק את התוצאה שלו.

0.4.1 דוגמה מלאה: האות בשני אתרי LIGO

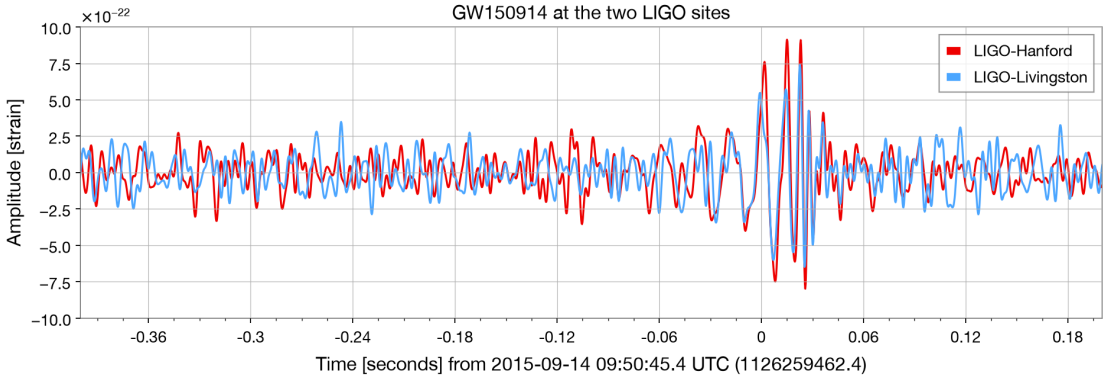
היה GW150914 הגילוי הישיר הראשון של גלי כבידה וההתמזגות הראשונה שנצפתה של שני חורים שחורים. נוכל להשוות את המעוות הציבורי מהאנפורד ומליווינגסטון (כלומר, בכמה עיוות המרחב עצמו בשני מקומות בארצות הברית) סביב האירוע באמצעות הדוגמה הבאה ב-Python, המבוססת על דוגמת הסינון של GWpy. נוריד 32 שניות מכל אתר, נפעיל אותו מסנן מעביר-פס של 50–250 הרץ ומסנני חריץ ב-60, 120 ו-180 הרץ, ונציג את 0.6 השניות המרכזיות.

הקוד שלמטה הוא תצוגה "כפי שהיא": טרם לימדנו את התחביר של Python או את הספרייה GWpy, ואף חלק ממנו אינו נבחן בסמסטר הזה. הוא כאן כדי לקבל מושג כיצד נראית זרימת העבודה ב-Python, ולהדגים עד כמה אקוסיסטם ה-Python נרחב. אפשר למצוא חבילות כמעט לכל דבר.

תבנית הבנה של רשימה כגון `[filter_design.notch(f, hdata.sample_rate) for f in (60, 120, 180)]` בונה רשומת מסנן אחת לכל תדר; הכוכב ב-`concatenate_zpks(bp, *notches)` פורש את הרשימה הזאת לארגומנטים נפרדים; ו-`lfilt *= -1` כופל את הסדרה במקומה במינוס אחת.

המסננים, הזזת הזמן והיפוך הסימן הם הפעולות שהופכות את שני העקבות לברי־השוואה.

```
from gwosc.datasets import event_gps
from gwpy.timeseries import TimeSeries
from gwpy.signal import filter_design
from gwpy.plot import Plot
t0 = event_gps("GW150914")
hdata = TimeSeries.fetch_open_data("H1", t0 - 16, t0 + 16)
ldata = TimeSeries.fetch_open_data("L1", t0 - 16, t0 + 16)
bp = filter_design.bandpass(50, 250, hdata.sample_rate)
notches = [filter_design.notch(f, hdata.sample_rate) for f in (60, 120,
180)]
zpk = filter_design.concatenate_zpks(bp, *notches)
hfilt = hdata.filter(zpk, filtfilt=True)
lfilt = ldata.filter(zpk, filtfilt=True)
lfilt.shift("6.9ms")
lfilt *= -1
plot = Plot(figsize=(12, 4))
ax = plot.gca()
ax.plot(hfilt, label="LIGO-Hanford", color="gwpy:ligo-hanford")
ax.plot(lfilt, label="LIGO-Livingston", color="gwpy:ligo-livingston")
ax.set(xlim=(t0 - 0.4, t0 + 0.2), ylim=(-1e-21, 1e-21),
       title="GW150914 at the two LIGO sites", ylabel="Amplitude [strain
]")
ax.set_xscale("seconds", epoch=t0)
ax.legend()
plot;
```



זהו איור ההשוואה שהפקודות עצמן החזירו. בכל סדרה שהורדה יש 131,072 דגימות בקצב 4,096 הרץ במשך 32 שניות; החלון הרחב מרחיק תופעות קצה של המסנן מחלון האירוע המוצג. סדרת ליווינגסטון מוזזת קדימה ב-6.9 מילישניות, הפרש זמן ההגעה שנמדד ודווח לאירוע זה, ומוכפלת ב--1 בגלל הכיוונים היחסיים של שני הגלאים. פעולות אלה מאפשרות להשוות ישירות את צורות הגל, והן מצוינות במפורש ולא מותאמות כאן מן הנתונים. נקודה-פסיק בסוף השורה מדכאת ייצוג כפול במחברת; בסקריפט עצמאי אפשר לקרוא ל-`plot.show()` או לשמור את האיור. סקריפט השחזור והגרסאות המתועדות נמצאים ב-`graphics/generate_gwpy_example.py` וב-`graphics/gw150914-gwpy-output.json`.

מה מודד הגרף של GW150914?

שני חורים שחורים התמזגו ופלטו גלי כבידה. גל חולף משנה את האורך היחסי של זרועות אינטרפרומטר: המעוות $h = \Delta L/L$ הוא חסר ממד. פלט הגלאי מכיל רעש וגם אות אפשרי. המסנן מעביר רק 50–250 הרץ, מסנני החריץ מדכאים הרמוניות צרות של רשת החשמל, וסינון קדימה ואחורה מונע

השהיית מופע מלאכותית. סינון לבדו אינו מוכיח גילוי. התאמה בין שני גלאים מרוחקים ועצמאיים לאחר תיקון הזמן והכיוון המתועד היא ראיה חזקה בהרבה מעקומה יחידה, אך ניתוח אמיתי גם מכמת את מובהקות הרעש ומשווה מודלים של צורת הגל.
האירוע נצפה ב־14 בספטמבר 2015 והוכרז ב־2016. שני גלאי LIGO, בהאנפורד ובליווינגסטון, צפו בו; Virgo לא פעל אז. פרס נובל לפיסיקה לשנת 2017 הוענק לריינר וייס, בארי בריש וקיפ תורן על תרומתם המכרעת לגלאי LIGO ולתצפית בגלי כבידה. כיול, רעש, סינון והנחות המודל מגבילים את הפרשנות.
נתוני GWOSC הציבוריים מופצים ברישיון CC BY 4.0; ראו https://gwosc.org/GW150914/.

0.5 למה ללמוד את שתיהן?

לאחר שנעבוד על חישוב פיסיקלי ב-Julia, חלק גדול ממנו יהיה מוכר כשנפגוש אותו ב-Python. המודל והיחידות נשארים כפי שהיו, גם כאשר הכתיב לשמירת נתונים או לקריאה לספרייה משתנה. מקרה שתשובתו ידועה נותן לנו בדיקה ראשונה בכל שפה, וגרפים ושאריות עוזרים לחקור את התוצאה. לימוד שפה שנייה מאפשר לנו לזהות את אותם רעיונות בסביבה נוספת.

נתחיל ב-Julia, משום שהכתיב הנומרי שלה מאפשר לעקוב מקרוב אחר המבנה של חישוב פיסיקלי. בהמשך, Python תפתח לנו גישה לאקוסיסטם הגדול יותר שנפגוש במעבדות, בקבוצות מחקר ובתוכנה מדעית ציבורית. בסיום הקורס נרגיש בנוח לקרוא קוד בשתי השפות, לבחור כלי למשימה מסוימת ולהסביר את הפיסיקה שמאחורי החישוב.

0.6 כיצד שנת הלימודים בנויה?

סמסטר א הוא בעיקרו סמסטר של Julia. נתחיל מיד בתהליך מעבדתי מסופק לבדיקת מדידות, התאמת ישר, שרטוט שאריות ופירוש פרמטרים עם יחידות; עבודה עצמאית עם קבצים וטבלאות תגיע בשבוע 7. לאחר מכן נבנה את יסודות התכנות ואת השיטות הנומריות הדרושים לביצוע ניתוחים כאלה באופן עצמאי. בשבוע 12 חקירת טיכו ברהה–קפלר תשלב את הכישורים האלה בניתוח אחד שניתן לשחזר.

בסמסטר ב נציג את Python באמצעות העברה של שיטות שכבר הבנו, ולאחר מכן נרחיב אותן לבעיות ערכים עצמיים, סימולציות מונטה קרלו, התאמה לא לינארית והתאמה עם אי ודאות, ניתוח פורייה, משוואות דיפרנציאליות חלקיות, ביצועים, למידת מכונה ותהליכים סימבוליים. תרגילי השבוע מספקים תרגול נדרש ובדיקת מוכנות; בהערכה ממוחשבת ניתן משקל גדול יותר לפתרון עצמאי של בעיה לא מוכרת ולהסבר המשמעות הפיסיקלית של התוצאה.

0.7 מה נלמד?

כדי להפוך בעיה פיסיקלית לתוכנית, נחליט תחילה כיצד החישוב מתקדם. מאילו ערכים נתחיל? מה משתנה בכל צעד, ומתי נעצור? נלמד לבטא את הבחירות האלה בעזרת חשיבה אלגוריתמית, לבחור ייצוג לנתונים ולפרק את החישוב לצעדים שאפשר לעקוב אחריהם ולבדוק. מקרים פשוטים וציפיות פיסיקליות יעזרו לנו להבין את הפלט.

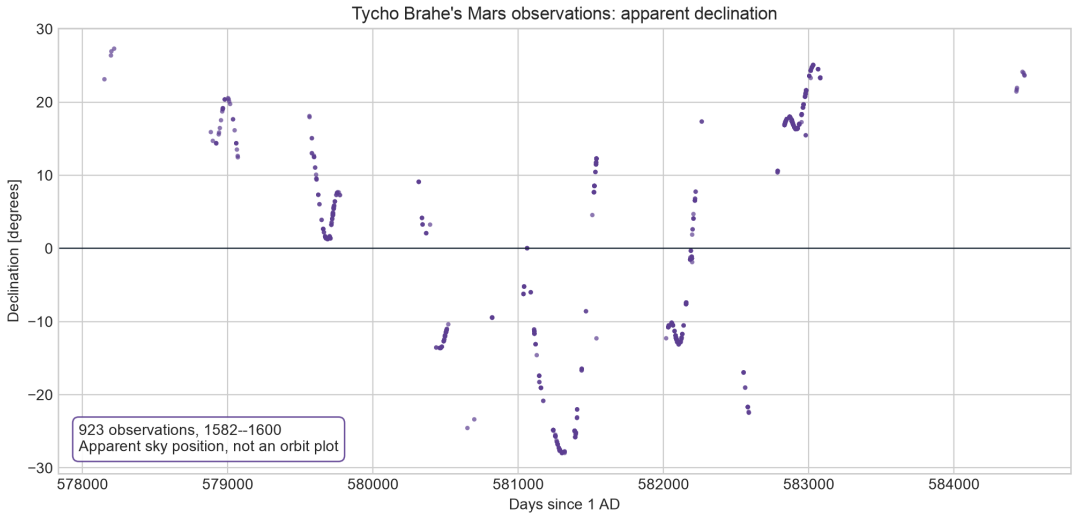
לאחר מכן נשתמש בהרגלים אלה כדי:

- לקרוא נתונים ניסיוניים ותצפיתיים מקבצים;
- להציג נתונים, מודלים, שאריות ואי ודאות;
- להתאים מודלים לינאריים ולא לינאריים ולפרש את הפרמטרים שלהם;
- לפתור משוואות, לחשב אינטגרלים ולפתור משוואות דיפרנציאליות רגילות נומרית;
- לעבוד עם וקטורים, מטריצות, התמרות פורייה, תהליכים אקראיים ומשוואות דיפרנציאליות חלקיות פשוטות;
- לכתוב קוד מדעי קריא, ניתן לבדיקה וניתן לשחזור ב-Julia וב-Python.

0.8 טיזר: לשחזר את ניתוח מאדים של קפלר

לקראת סוף הסמסטר הראשון נשתמש בתצפיות דיגיטליות הקשורות לתוכנית המדידות של טיכו ברהה על מאדים, כדי לשחזר גרסה של הטיעון של קפלר. מערך נתונים זה יהפוך לראיה לבחינת החוק הראשון של קפלר: מסלולי כוכבי הלכת הם אליפסות שהשמש באחד ממוקדיהן. ניקח תצפיות מתאריכים שונים, נהפוך מיקומים זוויתיים מתועדים לתיאור גיאומטרי משותף, נתאים מודלים מתחרים של מסלול, ונראה באופן כמותי מדוע אליפסה מצליחה במקום שבו מעגל נכשל.

הגרף הראשון מציג 923 מדידות של הנטייה הנראית של מאדים, הזווית שלו צפונה או דרומה בשמים, מן השנים 1582–1600. כבר עכשיו נוכל לחפש תבניות בשינוי הזווית לאורך הזמן. כדי לקשר אותן לצורת המסלול, נצטרך להביא בחשבון את גאומטריית התצפית ולהשוות תחזיות של מודלים מעגליים ואליפטיים. גרף הנטייה לבדו משאיר את השאלה הזאת פתוחה.



טיכו רשם תאריכים ומיקומים נמדדים בטבלאות. הצגתם בגרף תיתן לנו מבט ראשון על התצפיות; בניית מודל של המסלול תעזור להבין מה יוצר את התבנית. בדרך ניעזר ברעיונות מן הקורסים שאתם לומדים במקביל:

- **מכניקה:** כוחות מרכזיים, תנועה מסלולית והפירוש הפיסיקלי של אליפסה שהשמש באחד ממוקדיה;
- **מתמטיקה:** וקטורים, המרות קואורדינטות, חתכי חרוט, מטריצות ושיטת מינימום ריבועים (linear least squares);
- **מעבדה:** אי ודאות, שאריות, מקור הנתונים וההבדל בין עקום שנראה טוב לבין מסקנה הנתמכת בראיות.

כשנגיע לחקירה הזאת, יהיו בידינו הכלים לכתוב את הניתוח בעצמנו ולהשתמש בתצפיות היסטוריות כדי לבחון את הטענה האמפירית המרכזית של החוק הראשון של קפלר. בינתיים, הגרף נותן לנו שאלה שתלווה אותנו לאורך הסמסטר. את הגאומטריה ואת שיטות ההתאמה נפתח כשנהיה מוכנים להשתמש בהן.

0.8.1 קבלת מאגר הקורס

מאגר הסטודנטים הוא `nshaviv/physprog`. נפתח מסוף בתיקיית אב לבחירתנו ונריץ:

```
git clone https://github.com/nshaviv/physprog.git
cd physprog
```

אם Git אינו מותקן, נפתח את עמוד המאגר ונשתמש ב-Code → Download ZIP, נחלץ את הארכיון ונפתח את התיקייה; שאר השלבים זהים.

Git יוצר את תיקיית הקורס המקומית. נריץ את פקודות הסביבה משורש המאגר החדש ונפתח אותו ב-VS Code. קובץ המבוא יציין אילו הרצאות, מטלות ופתרונות פורסמו. הוראות התקנה ועדכון מפורטות נמצאות ב-`STUDENT_SETUP.md` שבשורש המאגר. אפשר לקרוא את המחברות מפלט שמור; פתיחתן

להרצה ב-VS Code עם הרחבת Jupyter או ב-JupyterLab היא רשות. המיקום במחשב האישי אינו צריך להיות זהה למיקום במחשב המרצה.

0.9 סביבת הקורס: התקנה חד־פעמית והרצה ניתנת לשחזור

נתקין את Julia 1.12 לפי ההוראות ב-https://julialang.org/install/; נתקין את Visual Studio Code ואת ההרחבה הרשמית של Julia לפי https://www.julia-vscode.org/docs/stable/gettingstarted/. במסוף חדש נוודא שהפקודה שנמצאת בנתיב היא ה-Julia שהתכוונו אליה:

```
julia --version
```

```
julia version 1.12.6
```

נוודא שהפלט מתחיל ב-julia version 1.12. אם מופיעה גרסה אחרת, נפעיל מחדש את המסוף לאחר ההתקנה ונבדוק איזו התקנה של Julia הוא מוצא.

נפתח ב-VS Code את תיקיית השורש של המאגר. זוהי התיקייה שמכילה את Project.toml לאחר השכפול. נריץ את הפקודות שלמטה מתוך תיקיית השורש הזאת. האפשרות –project=. מפעילה את סביבת הקורס המתועדת; האפשרות e- שאחריה אומרת ל-Julia להריץ את הארגומנט שלה.

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Julia קוראת את Project.toml ואת המניפסט המתועד, מורידה חבילות מתועדות שחסרות, ומדווחת על התקנה או קומפילציה מוקדמת של חבילות. נריץ זאת פעם אחת כאשר יש חיבור לרשת. לא נשתמש ב-Pkg.add או ב-Pkg.update בעבודת הקורס: פקודות אלה ישנו סביבה במקום להשתמש בסביבה המתועדת.

לאחר מכן נריץ את פקודת בדיקת הקורס עם אותה בחירת פרויקט:

```
julia --project=. verify_environment.jl
```

```
Course Julia environment verified.
```

הסקריפט בודק את הסביבה המוגדרת ומסתיים בהודעת ההצלחה שלו. הוא עשוי ליצור קובצי גרפים זמניים מחוץ למאגר. אם הוא מדווח על חבילה חסרה, נסגור חלונות Julia REPL ישנים, נוודא ש-VS Code בחר את סביבת הקורס, ונחזור על פקודת ההתקנה.

ב-VS Code נבחר את אותה סביבת הקורס עבור הרחבת Julia. קובץ שמוריץ מתוך העורך שולח ערכים וטקסט של println אל Julia REPL המשולב. העורך הוא המקום שבו אנו כותבים קוד; ה-REPL הוא הפעלה חיה של Julia שמעריכה אותו. ההבחנה הזאת תועיל בפרק 1.

רשות: הרצת המחברות ב-VS Code

כל מחברת כוללת כבר את הפלט השמור שלה, כך שאפשר לקרוא אותה בלי להריץ דבר. כדי להריץ מחברת באופן אינטראקטיבי:

1. ב-VS Code נתקין את הרחבת Jupyter (הרחבת Julia נשארת לקבצי jl.).
2. נפתח את שורש המאגר, ואז נפתח קובץ ipynb. ונשתמש ב-Select Kernel כדי לבחור `Julia 1.12` עבור מחברות ההרצאה והתרגול, או את סביבת Python של הקורס עבור מחברות ה-Python של פרק 0.
3. נריץ תאים בזה אחר זה, או נשתמש ב-Restart Kernel and Run All כדי לשחזר פרק מהתחלה נקייה.

אותן מחברות נפתחות גם ב-JupyterLab. הרצת המחברות היא אפשרות, ולא חובה.



1. חישובים ראשונים ב-Julia והתאמה מודרכת לחוק אוהם

נתחיל חישוב מדעי בניבוי של מה שאמור לקרות, בהצגה גלויה של החישוב ובשאלה אם לתוצאה יש את היחידות והמשמעות הנכונות. היום נשתמש ב-Julia כמחשבון, ואחר כך נעבור למתכון נתון שמתחיל בנתונים, ממשיך בהתאמה ומסתיים באיור. המתכון מכיל וקטורים ופקודות שרטוט שנלמד בנפרד בהמשך.

1.1 היכן רואים תוצאה?

ב-REPL של Julia, הקלדת ביטוי מחשבת אותו ומציגה את ערכו. בקובץ VS Code ההערכה יכולה גם היא להציג ערך בחלון תוצאה. התצוגה נוחה, אך אינה חלק עמיד מדוח של סקריפט. println שולחת טקסט במפורש ל-REPL, לטרמינל או לפלט הסקריפט; נשתמש בה כאשר הקורא חייב לראות תוצאה.

```
1 + 1
println(1 + 1)
```

```
2
```

הערכת השורה הראשונה ב-REPL או בתצוגת עורך מציגה 2. השורה השנייה מדפיסה 2 לפלט ה-REPL. סקריפט עשוי להעריך ביטויים רבים בלי להציגם, ולכן נשתמש ב-println לדוח טקסטואלי מכוון.

1.2 Julia כמחשבון

ל-Julia אריתמטיקה מוכרת, סוגריים, חזקות ופונקציות מתמטיות. רוב הביטויים שלהלן הם קריאות לפונקציה: שם פונקציה ואחריו סוגריים, כגון $\exp(1)$. הקריאה מבקשת מהפונקציה הנקראת לפעול על הערכים שבסוגריים ולהחזיר תוצאה. הארגומנטים של sin ו-exp הם ברדיאנים כאשר הם מייצגים זוויות. $\mathrm{mod}(a,b)$ היא השארית לאחר חלוקה ב-b; $\mathrm{div}(a,b)$ היא חלוקת מנה שלמה, שמשליכה את השארית עבור מספרים שלמים חיוביים.

```
println(exp(1))
println(sin(pi / 2))
println(mod(17, 5))
println(div(17, 5))
```

```
2.718281828459045
1.0
2
3
```

כאן $2 + 3*5 = 17$; המנה היא 3 והשארית 2.

אפשר להקליד סמלים מתמטיים ב-REPL וב-VS Code. הקלידו `\pi<Tab>` כדי לקבל π. בביטוי `2pi` או 2π, Julia מפרשת את המקדם המספרי ככפל ב-pi; זה אינו שם משתנה אחד. בין שני שמות משתנים יש לכתוב את סימן הכפל במפורש, כגון `speed * mass`.

```
println(2pi)
println((2 + 3) * 4)
println(2^3)
```

```
6.283185307179586
20
8
```

1.3 שמות לערכים לפני הפיזיקה

הצבה כגון `x = 2` מצמידה את השם שמשמאל לערך המחושב מימין. זו הוראה, ולא הקביעה האלגברית $x = 2$ לכל הזמנים. לאחר מכן אפשר להשתמש בשמות כדי לחשוף חישוב.

```
x = 2
y = 3
println(x + y)
println(x^2 + y^2)
```

```
5
13
```

נשמור שמות נקיים מסיומות יחידה ונתעד כל יחידה בהערה קצרה בסוף השורה במקום שבו הערך מוגדר, למשל `voltage = 2.0  # V`. Julia אינה מסיקה יחידות מן ההערה ואינה אוכפת אותן, ולכן ההערה והתיעוד של הפונקציה נושאים אותן.

1.4 תהליך מודרך ממדידה למודל

המתכון הבא משחזר את ההסבר המלא של הניתוח הראשון שלנו. אנו מספקים את שלבי קריאת הקובץ, הווקטורים והשרטוט. כעת אנחנו אמורים להריץ ולפרש את השלבים; עבודה עצמאית עם CSV/DataFrames שייכת לשבוע 7, ושליטה בהתאמה לשבוע 8. הגזירה המתמטית היא רקע לבחירה, נפרד מהבונוס המזכה בשליש נקודה. איננו מגדירים פונקציות לפני שבוע 5.

נפתח את המחברת המצורפת לשבוע 1, או נריץ תוכנית שמורה מתיקייה שמכילה את `ohm_law.csv`. סביבת הקורס המשותפת כבר מצהירה על CSV, DataFrames ו-Plots; ההתקנה מתוארכת בפרק 0.

מהו קובץ CSV?

קובץ CSV (ראשי תיבות של comma-separated values) שומר טבלה כטקסט פשוט. השורה הראשונה מכילה בדרך כלל את שמות העמודות; כל שורה אחריה היא שורה בטבלה, ופסיקים מפרידים בין הערכים. למשל,

```
current_A,voltage_V
0.10,0.52
0.20,1.01
```

הוא קובץ עם שתי עמודות: זרם באמפר ומתח בוולט. אפשר לייצא טבלה לקובץ CSV מגיליון אלקטרוני או מתוכנת מעבדה. טקסט פשוט שימושי משום שאדם יכול לבדוק אותו ותוכנות רבות יכולות לקרוא אותו. בקורס נשתמש ב-CSV.jl לקריאת הקובץ וב-DataFrames.jl לארגון הטבלה.

מוליך אוהמי

במוליך אוהמי במצב קבוע בקירוב ובתחום שדה חלש ותגובה לינארית, מתקיים $V = RI$. נסמן מתח מופעל V בוולט וזרם חיובי I באמפר בכיוון החיובי שנבחר; ל-$R = V/I$ יחידות Ω, ולמוליכות $G = 1/R$ יחידות סימנס. פיזור (scattering) מבסס סחיפה קבועה ועוזר לקבע את ההתנגדות, אך התנגשויות תכופות לבדן אינן מבטיחות לינאריות. חימום, מגעים, שדות חזקים והתקנים לא-אוהמיים יכולים לפסול את המודל. הבדיקה האידיאלית שלו היא $I = 0 \Rightarrow V = 0$.

רקע מתמטי: מודל של ישר

במודל $y = ax + b$, a הוא השיפוע ו-b החיתוך. השיפוע מתאר בכמה y משתנה כאשר x גדל ביחידה אחת; החיתוך הוא ערך המודל ב-$x = 0$.

כיצד נציג רעיון חדש בתכנות

לפני שנשתמש במבנה חדש נשאל מדוע הוא דרוש, נגדיר מהו, נציג את הצורה הכללית שלו ב-Julia, וננסה את הדוגמה השימושית הקטנה ביותר. לאחר מכן נשתמש בו בחישוב עשיר יותר ולבסוף נקשר אותו למשימה מדעית. גם בקריאת קוד נפעל בסדר הזה: מטרה, צורה, מקרה פשוט ויישום.

דרכים להריץ Julia בשבוע 1

- **טרמינל מערכת:** פתחו Terminal, PowerShell או מעטפת Linux והקלידו julia. ייפתח REPL, שבו 1+1 מחושב מיד.
- **טרמינל בתוך VS Code:** בחרו Terminal > New Terminal והקלידו julia. זהו אותו REPL, לצד קבצי הקורס.
- **REPL של הרחבת Julia:** פתחו את Command Palette ובחרו Julia: Start REPL. כאשר קובץ jl. פתוח, Shift+Enter שולח את השורה או הקטע הנבחר ל-REPL ויפעיל אותו אם צריך.
- **תוכנית שמורה מלאה:** בחרו Run > Run Without Debugging כדי להריץ את קובץ ה-jl. הפעיל מתחילתו ועד סופו. זו הדרך הרגילה לעבודה להגשה, משום שהניתוח נשמר וניתן לשחזור.

בהמשך נלמד מחברות, שבהן תאים אינטראקטיביים יותר. אין לבלבל בין תא במחברת לבין ניתוח שמור: בשניהם דרושים קלטים, קוד, גרפים ומסקנות ברורים.

1.4.1 לבנות את הניתוח, צעד גלוי אחר צעד

צעד 1: טעינת הכלים. השורה הראשונה בסקריפט מציינת באילו חבילות מותקנות התוכנית תשתמש.

```
using CSV, DataFrames, Plots
```

המילה using הופכת את הכלים הציבוריים של החבילות לזמינים בקובץ זה. כאן CSV קוראת קובץ טקסט, DataFrames מספקת מבנה של טבלה, ו-Plots מייצרת איורים. זו אינה עדיין פעולה חישובית: התוכנית מצהירה אילו כלים היא צריכה.

צעד 2: טעינת הטבלה ובדיקה ראשונה שלה. השורות הבאות פותחות את הקובץ ומציגות את חמש השורות הראשונות.

לפני כן דרושים שני דפוסי קוד בסיסיים. קריאה לפונקציה, כפי שראינו עם exp ו-sin, מבקשת מכלי בעל שם לבצע פעולה. צורתה הכללית היא function_name(input1, input2); למשל, println("ready") קוראת ל-println עם קלט טקסטואלי אחד. הצבה נותנת לתוצאה שם שאפשר להשתמש בו שוב. צורתה הכללית היא name = expression; תחילה Julia מחשבת את צד ימין ולאחר מכן שומרת את התוצאה בשם שמשמאל. הדוגמה הקטנה ביותר היא x = 2. הבלוק הבא משלב את שני הדפוסים עם קובץ נתונים אמיתי.

הקלט השני של CSV.read, כלומר DataFrame, נותן שם לטיפוס הטבלה שאנו רוצים לקבל: טבלה בעלת עמודות בעלות שם. הפונקציה העוזרת first(data, 5) מחזירה את חמש השורות הראשונות של

הטבלה, כדי שנוכל לבדוק אותן.

```
root = get(ENV, "PHYSPROG_HOME", pwd())    # the course folder
candidates = [joinpath(root, "assets", "data", "ohm_law.csv"),
              joinpath(root, "reveal", "week01", "ohm_law.csv")]
datafile = candidates[findfirst(isfile, candidates)]
data = CSV.read(datafile, DataFrame)
println(first(data, 5))
```

עם הקובץ שסופק, חמש שורות הנתונים המודפסות הן:

שורה	current_A	voltage_V
1	0.1	0.52
2	0.2	1.01
3	0.3	1.52
4	0.4	2.0
5	0.5	2.49

קוראים את CSV.read(datafile, DataFrame) משמאל לימין. CSV.read פירושו "השתמשו בפונקציה read שמספקת החבילה CSV". הקלט הראשון הוא שם הקובץ; הקלט השני מבקש ליצור DataFrame, כלומר טבלה בעלת עמודות בעלות שם. סימן השוויון שומר את הטבלה בשם data. השורה הבאה לוקחת את חמש השורות הראשונות ומדפיסה אותן ב-REPL או בטרמינל. זו בדיקה מדעית ראשונה: האם שמות העמודות נכונים, האם המספרים סבירים, והאם פתחנו את הקובץ שהתכוונו אליו?

צעד 3: מתן שמות לגדלים הפיסיקליים. מוציאים את שתי העמודות לשמות משתנים נקיים מיחידות, שהגדרותיהם מתעדות את היחידות בהערה קצרה.

DataFrame נותנת שם לכל עמודה. הצורה table.column_name בוחרת עמודה שלמה; למשל, data.current_A בוחרת את המדידות שנשמרו תחת הכותרת current_A. נציב את שתי העמודות שנבחרו בשמות בעלי משמעות:

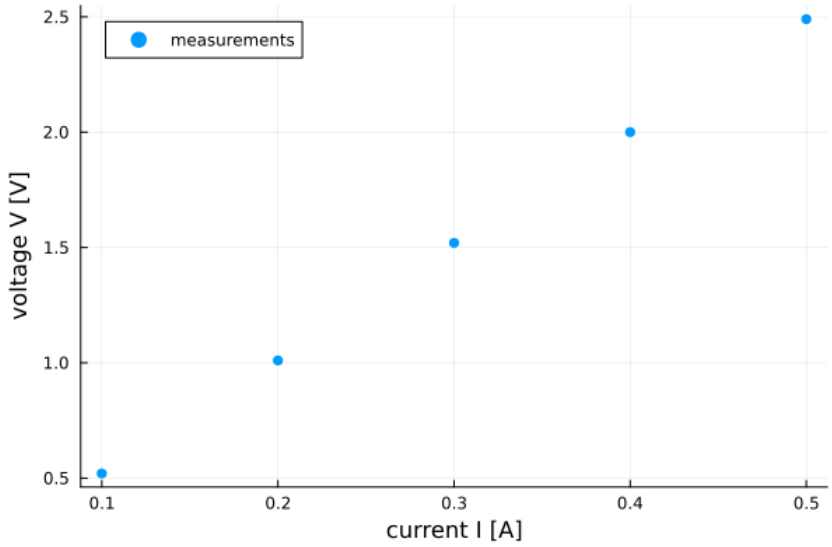
```
current = data.current_A   # A
voltage = data.voltage_V   # V
```

השמות current ו-voltage הם משתנים: תוויות שמאפשרות להוראות הבאות להתייחס לערכים שהתוכנית שומרת. סימן השוויון מציב במשתנה שמשמאל את הערך שמימין; הוא אינו מציין זהות מתמטית קבועה. הנקודה ב-data.current_A בוחרת את העמודה ששמה current_A. אנו משאירים את המזהים נקיים מיחידות וקוראים את היחידה מן ההערה שבסוף השורה, שאותה Julia מתעלמת. יש לתעד יחידות במקום שבו הערכים מוגדרים, במחרוזות התיעוד או בתיעוד הממשק של פונקציות, ובאתרי קריאה שבהם היחידות עלולות להיות עמומות. כאן אנו משתמשים במספרים שלמים לספירה ובמספרים עשרוניים למדידות בלי לפתח עדיין את מערכת הטיפוסים של Julia; בשבוע 2 ניתן שמות לטיפוסים הפשוטים ונבדוק אותם במפורש.

צעד 4: לראות את הנתונים. לפני כל התאמה, מציירים את המדידות.

וקטור הוא רשימה סדורה שנכתבת בתוך סוגריים מרובעים, למשל $[0, 1]$. גרף פיזור מייצג כל זוג ערכים משני וקטורים באמצעות נקודה אחת, ולכן מאפשר לראות מגמה ומדידות חריגות לפני שסומכים על מודל. צורת הקריאה הבסיסית היא scatter(x_values, y_values). למשל, scatter([0, 1], [0, 1]) מצייר שתי נקודות. הגדרות אופציונליות בעלות שם נכתבות בצורה keyword=value; הן מוסיפות תוויות או משנות את המראה בלי לשנות את הנתונים. למשל, markerstrokewidth=0 מסיר את המסגרת הנמתחת סביב כל סמן, כך שנקודות חופפות נשארות קריאות. כעת ניישם את הצורה על הזרם והמתח:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
```



איור 1.1: הפלט הגרפי של הקריאה ל-scatter: המתח הנמדד גדל בקירוב לינארי עם הזרם.

שני הקלטים הראשונים של scatter הם הקואורדינטות האופקיות והאנכיות. הקלטים שאחריהם הם ארגומנטים בעלי שם: לכל אחד שם, כגון xlabel, אחריו סימן שוויון וערך. הם שולטים במראה האיור. תוויות ומקרא אינם קישוט; הם מסבירים לקורא מה נמדד ובאילו יחידות.

הגרף הוא חלק מהחישוב

אין לסמוך על התאמה שלא ראינו בגרף. תשובה מספרית יכולה להסתיר נקודה חריגה, מגמה לא לינארית, טעות ביחידות או עמודות שהוחלפו.

צעד 5: התאמת ישר לפי ההגדרה. עבור ישר ניסיון $V = RI + V_0$, השארית במדידה i היא אי-ההתאמה האנכית

$$r_i = V_i - (RI_i + V_0).$$

נבחר את הישר שממזער את סכום ריבועי אי-ההתאמות,

$$S(R, V_0) = \sum_{i=1}^{n} \left[V_i - (RI_i + V_0) \right]^2 .$$

העלאה בריבוע מונעת מאי-התאמות חיוביות ושליליות לבטל זו את זו, ומענישה אי-התאמה גדולה יותר מהתאמה קטנה. זוהי הגדרת ההתאמה הלינארית ללא שקלול: היא מתייחסת לכל המדידות כבעלות מהימנות שווה. בהמשך הקורס נלמד מינימיזציה. בשבוע 1 משתמשים בנוסחאות הישירות שמתקבלות מכך, ולא בממזער כללי או בשיטת מטריצות.

כדוגמה מדויקת ופשוטה, שלוש הנקודות $(0,1)$, $(1,3)$ ו-$(2,5)$ נמצאות כולן על הישר $y = 2x + 1$. שיפוע ההתאמה הוא 2, החיתוך הוא 1, וכל השאריות שוות לאפס. מדידות אמיתיות אינן נמצאות בדיוק על ישר אחד, ולכן חישוב חוק אוהם שלהלן הוא המקרה העשיר יותר.

נוסחאות ההתאמה הלינארית הישירות

נסמן ב-$\overline{I}$ את ממוצע הזרמים וב-$\overline{V}$ את ממוצע המתחים. השיפוע והחיתוך המותאמים הם

$$R = \frac{\sum_{i=1}^{n} (I_i - \overline{I})(V_i - \overline{V})}{\sum_{i=1}^{n} (I_i - \overline{I})^2}, \qquad V_0 = \overline{V} - R\overline{I}.$$

המונה חיובי כאשר הזרם והמתח נוטים לגדול יחד. המכנה מודד את פיזור ערכי הזרם סביב הממוצע.

התאמה אינה אפשרית אם כל ערכי הזרם זהים, משום שאז אין אפשרות לקבוע שיפוע. החיתוך נבחר כך שהישר המותאם יעבור דרך $(\overline{I}, \overline{V})$.

רקע מתמטי לבחירה: מאין מגיעות נוסחאות ההתאמה?

תיבה זו מיועדת למי שרוצה להקדים מעט; היא אינה נדרשת בשבוע 1. מחפשים מינימום של $S(R, V_0)$, התלויה בשני פרמטרים שניתן לשנות. במינימום חלק, שינוי קטן בכל אחד מן הפרמטרים אינו יכול להקטין את S, ולכן שתי הנגזרות החלקיות מתאפסות.
תחילה נגזור לפי החיתוך:

$$0 = \frac{\partial S}{\partial V_0} = -2\sum_{i=1}^{n} \left[V_i - (RI_i + V_0)\right].$$

חלוקה ב-$-2n$ וסידור מחדש נותנים $V_0 = \overline{V} - R\overline{I}$. לכן הישר המותאם עובר דרך נקודת הנתונים הממוצעת.
כעת נגזור לפי השיפוע R:

$$0 = \frac{\partial S}{\partial R} = -2\sum_{i=1}^{n} I_i \left[V_i - (RI_i + V_0)\right].$$

מציבים את הביטוי עבור V_0. השארית נעשית $(V_i - \overline{V}) - R(I_i - \overline{I})$. בעזרת $\sum_i (I_i - \overline{I}) = 0$ ו-$\sum_i (V_i - \overline{V}) = 0$, המשוואה הנותרת היא

$$\sum_{i=1}^{n} (I_i - \overline{I})(V_i - \overline{V}) = R \sum_{i=1}^{n} (I_i - \overline{I})^2.$$

פתרון המשוואה עבור R נותן את נוסחת השיפוע הישירה שמופיעה למעלה; הצבה של השיפוע ב-$V_0 = \overline{V} - R\overline{I}$ נותנת את החיתוך. בהמשך, עם העמקת ההתאמה בשבוע 8 והמזעור, מינימיזציה תהפוך את דרך החשיבה הזו לכלי קבוע בקורס.

יש להפעיל את הנוסחאות על כל וקטור המדידות באותה פעולה. נקודה על אופרטור חשבוני אומרת ל-Julia להפעיל את הפעולה על כל איבר בנפרד. הדפוס הכללי הוא operator. vector value. הסימן # מתחיל הערה: Julia מתעלמת משאר השורה, ואילו הקורא יכול להשתמש בה כהסבר. למשל,

```
values = [1, 2, 3]
squared = values .^ 2     # [1, 4, 9]
```

כעת נפעיל פעולות מנוקדות על כל מדידות הזרם והמתח. קודם כל שתי פעולות סיכום על וקטור: length(current) מחזירה את מספר המדידות, ו-sum(current) מחברת את איברי הווקטור. בשתיהן נשתמש כדי לחשב את הממוצעים. הקוד מממש את הנוסחאות ישירות, ולכן כל גודל נשאר גלוי:

```
n = length(current)
I_bar = sum(current) / n   # A
V_bar = sum(voltage) / n   # V
dI = current .- I_bar      # A
dV = voltage .- V_bar      # V
resistance = sum(dI .* dV) / sum(dI .^ 2)   # ohm
offset = V_bar - resistance * I_bar   # V
```

length(current) סופרת את המדידות ו-sum(...) מחברת את איברי הווקטור. הנקודה ב-.-, ב-.* ובו-.^. אומרת "הפעילו פעולה זו על איברים מתאימים". לכן dV .* dI מכיל מכפלה אחת $(I_i - \overline{I})(V_i - \overline{V})$ לכל מדידה, מוכנה לחיבור בעזרת sum. Julia מבדילה בכוונה בין פעולות איבר-איבר לבין פעולות מטריציוניות; כפל מטריצות רגיל אינו מוגדר עבור שני וקטורים של מדידות תואמות. בשבוע 1 משתמשים

בצורה המנוקדת כאשר החישוב אמור לפעול על כל מדידה בנפרד.

צעד 6: להציג את המודל על אותו גרף ולדווח על התוצאה.

אנו בונים את גרף המודל מהעתק חדש של נקודות המדידה, ואז מוסיפים לאותו איור את הישר המותאם. בצורה plot!(x, y) סימן הקריאה מציין שגרף קיים משתנה; לאחר מכן savefig("name.png") שומרת את הגרף הנוכחי. קריאה ל-plot() בלי נתונים מתחילה גרף חדש וריק. כדי לדווח מספרים, $(expression) משלב ערך מחושב בתוך מחרוזת, ו-round(value, digits=3) מגבילה את מספר הספרות המוצגות אחרי הנקודה. דוגמה קטנה לכל אחת מן הצורות, בגרף נפרד משלה, היא:

```
plot()
plot!([0, 1], [1, 0])
savefig("example.png")
rounded = round(1.2345, digits=2)
println("value = $(rounded)")
```

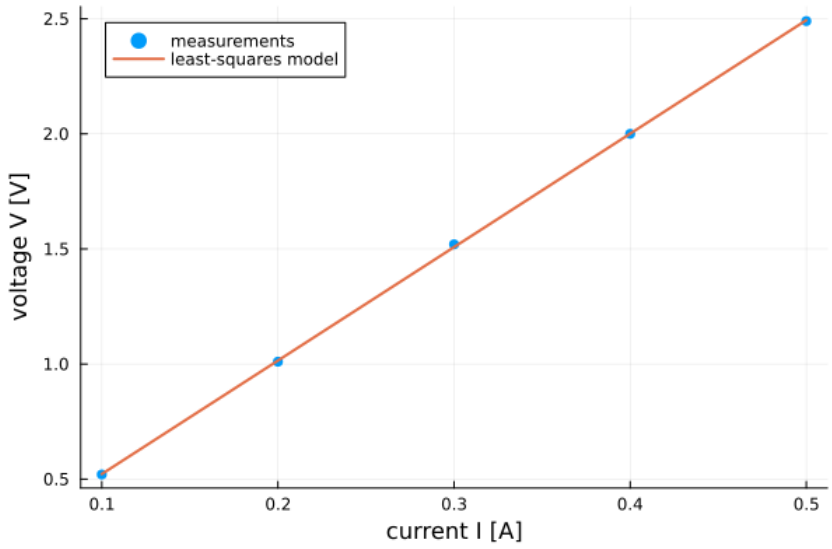
```
value = 1.23
```

בלוק הדיווח המדעי המלא יוצר מחדש את פיזור המדידות ומוסיף את המודל המותאם, כך שהישר הצעצועי אינו יכול להישאר באיור. על קו המודל, linewidth=2 קובע את רוחב הקו:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
plot!(current, resistance .* current .+ offset,
    label="least-squares model", linewidth=2)
savefig("ohm_law_fit.png")

println("R = $(round(resistance, digits=3)) ohm")
println("offset = $(round(offset, digits=3)) V")
```

```
R = 4.93 ohm
offset = 0.029 V
```

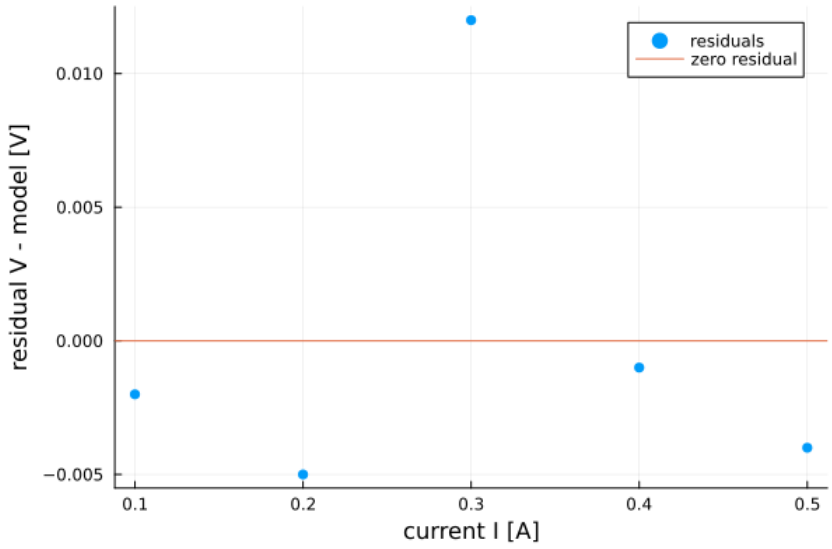


איור 1.2: הפלט הגרפי לאחר plot!: מודל מינימום הריבועים מצויר על המדידות.

הנקודה ב-current *. resistance מכפילה את ההתנגדות הסקלרית בכל ערך זרם. הנקודה ב-+. offset מוסיפה את החיתוך לכל תוצאה. סימן הקריאה ב-plot! פירושו "הוסיפו לגרף הקיים" במקום ליצור גרף חדש. הקריאה הבאה ל-savefig שומרת את הגרף הנוכחי כקובץ PNG בתיקיית העבודה.

צעד 7: בדיקת השאריות. ישר מותאם הוא מודל, ולא הוכחה שהמודל טוב. מציירים את השאריות כדי לבדוק אם הן מתפזרות סביב אפס ללא דפוס בולט. הצורה המשנה !hline מוסיפה קו אופקי לגרף הנוכחי, ולכן נשתמש בה לקו ייחוס באפס:

```
residual = voltage .- (resistance .* current .+ offset)   # V
scatter(current, residual,
    xlabel="current I [A]", ylabel="residual V - model [V]",
    label="residuals", markerstrokewidth=0)
hline!([0], label="zero residual")
savefig("ohm_law_residuals.png")
```



איור 1.3: הפלט הגרפי של חישוב השאריות: כל השאריות קטנות ונמצאות משני צדי האפס.

הקריאה השנייה ל-savefig שומרת את גרף השאריות, שהוא הגרף הנוכחי באותו רגע. אם השאריות יוצרות עקומה שיטתית מעל ומתחת לאפס, ייתכן שישר אינו המודל המתאים. אם שארית אחת גדולה בהרבה מהאחרות, בודקים את המדידה במקום לסמוך על ההתאמה בשקט. בשלב זה יש לנו תוכנית מדעית קטנה אך שלמה: היא קוראת נתונים, בודקת אותם, שומרת איורים, מחשבת פרמטרים של מודל ומדווחת על משמעותם עם יחידות.

פיזור מדידות, שגיאת מודל וספרות מדווחות

פיזור מדידות הוא השינוי בין תצפיות חוזרות הנובע מרזולוציית המכשיר ומתנאי הניסוי. שגיאת מודל היא אי התאמה שיטתית בין המודל שנבחר לבין המערכת הפיסיקלית; דפוס עקום בשאריות הוא ראיה לאי התאמה כזו. שני אלה שונים מעיגול תצוגה. הדפסת שלוש ספרות אחרי הנקודה יוצרת פלט אחיד, אך אינה מוכיחה שכל שלוש הספרות משמעותיות פיסיקלית. מספר הספרות המשמעותיות המוצדק צריך לנבוע לבסוף מאי ודאות המדידה, שבה נטפל בהרחבה בהמשך.

תרגול מונחה של תהליך העבודה נמצא בפרק 1 של חוברת התרגול, נתוני נפילה חופשית.

רקע פיסיקלי: יחידות בפרמטרים מותאמים

אם y נמדד בוולטים ו-x באמפרים, לשיפוע יש יחידות וולט לאמפר, כלומר אוהם. לחיתוך יש אותן יחידות כמו y. בדיקת יחידות היא דרך פשוטה לגלות טעויות בהתאמה. בהמשך, חבילה כגון Unitful.jl תוכל לשמור יחידות יחד עם הערכים בקוד; החשיבה הפיסיקלית נשארת אותה חשיבה.

1.5 תרגול: מדידת קבוע קפיץ

נעביר את אותו ניתוח לקפיץ. זוגות המדידות הם התארכות x במטרים וכוח מופעל F בניוטון; נתאים את המודל $F = kx + F_0$. כאן, בניגוד לגרף שבו ההתארכות בציר האנכי, השיפוע הוא ישירות קבוע הקפיץ k ביחידות ניוטון למטר. נצפה מראש לשיפוע חיובי ונבדוק שלחותך יש יחידות כוח.

לפני המפגש נפתח את מחברת התרגול לשבוע 1 ונאתר את הנתונים ואת תאי ההתאמה המסופקים. בעשרים הדקות הראשונות נבחן את הצירים, נעקוב אחר התאמה אחת ונפרש את השיפוע, ההיסט ושארית אחת. לאחר מכן נתחיל את מטלת הנפילה החופשית בעזרת צוות ההוראה. נקודת בדיקה מועילה היא להסביר מדוע החלפת הצירים משנה את יחידות השיפוע ואת משמעותו; בנתונים רועשים, התאמה מחודשת בצירים ההפוכים אינה בהכרח הופכי השיפוע הקודם.

1.6 מטלת החובה: הסקת תאוצת הכובד ממדידות נפילה חופשית

נבדוק אם מרחקי הנפילה מתאימים לשחרור ממנוחה בכבידה אחידה בקירוב. נגדיר מרחק חיובי כלפי מטה, ולכן $y(t) = \tfrac{1}{2} g t^2 + b$, כאשר $g > 0$ ביחידות מטר לשנייה בריבוע ו-b במטרים. הפעולה המסופקת `t_s .^ 2` הופכת את ריבוע הזמן לקואורדינטה האופקית. נתאים $y = at^2 + b$ ונפרש $g = 2a$: הגורם שתיים נובע ממשוואת התנועה ולא ממוסכמה של התוכנה.

נעבוד בשלבים:

1. נבחן את `free_fall.csv` המסופק: נזהה את עמודות הזמן והמרחק, את יחידותיהן ואת מספר המדידות.
2. נחזה את צורת הגרפים של y מול t ושל y מול t^2 לפני ההתאמה. ניישם את המתכון ונשמור את התחזיות ואת השאריות, מדידה פחות תחזית.
3. נכין איור אחד: הנקודות הנמדדות כסמנים עם המודל המותאם מעליהן, וכל ציר מסומן בכמותו וביחידותיו.
4. נשמור את האיור כ-PNG או PDF ונדווח על g ועל ההיסט. נסביר אם השאריות תומכות במודל בתחום המדידות ונציין הנחה שההתאמה לבדה אינה מוכיחה.
5. נגיש את הסקריפט ואת האיור בעמוד ה-Moodle של הקורס, כפי שמפורט בדף המטלה.

נגיש סקריפט שלם יחד עם האיור; בשבוע 1 הגדרת פונקציה אינה חלק מהעבודה. המטלה הראשונה נבדקת ידנית ב-Moodle, עבור/לא עבור. כדאי להשאיר את דף המטלה לצידנו, מפני שבו מפורטים העמודות הנדרשות, האיור והגדלים שיש לדווח. עיגול התוצאה לצורך הצגה אינו צריך לעגל את הערכים המשמשים לחישוב.

1.7 בונוס לבחירה: גרף שמעביר תוצאה מדעית

הבונוס הוא רשות, אינו תנאי להמשך ואינו נבחן. השלמתו מזכה ב**שליש נקודה בציון הסופי**.

לאחר קריאה עצמאית של 10–15 דקות נשפר גרף מינימלי מסופק של חוק אוהם. כאן המתח בציר האופקי והזרם בציר האנכי, ולכן שיפוע המודל הוא מוליכות בסימנס ולא התנגדות באוהם. נזהה את סדרת הזרם הנמדד ואת סדרת הזרם המותאם לפני שינוי העיצוב.

נוסיף את הכותרת, תוויות הצירים והיחידות, המקרא, סמני המדידות, קו המודל, המסגרת וגודל האיור שנקבעו, ונייצא PNG ו-PDF. קטע החישוב מספק סדרות מאומתות ותואמות; התמונות נבדקות בנפרד לפי הרשימה המפורשת. נוודא ששינוי תווית או סגנון קו אינו משנה מדידה. דף הבונוס מציין את הטקסטים ושמות הקבצים המדויקים, כך שהבדיקה עוסקת בשלמות מדעית ולא בטעם אישי או בהשוואת פיקסלים.

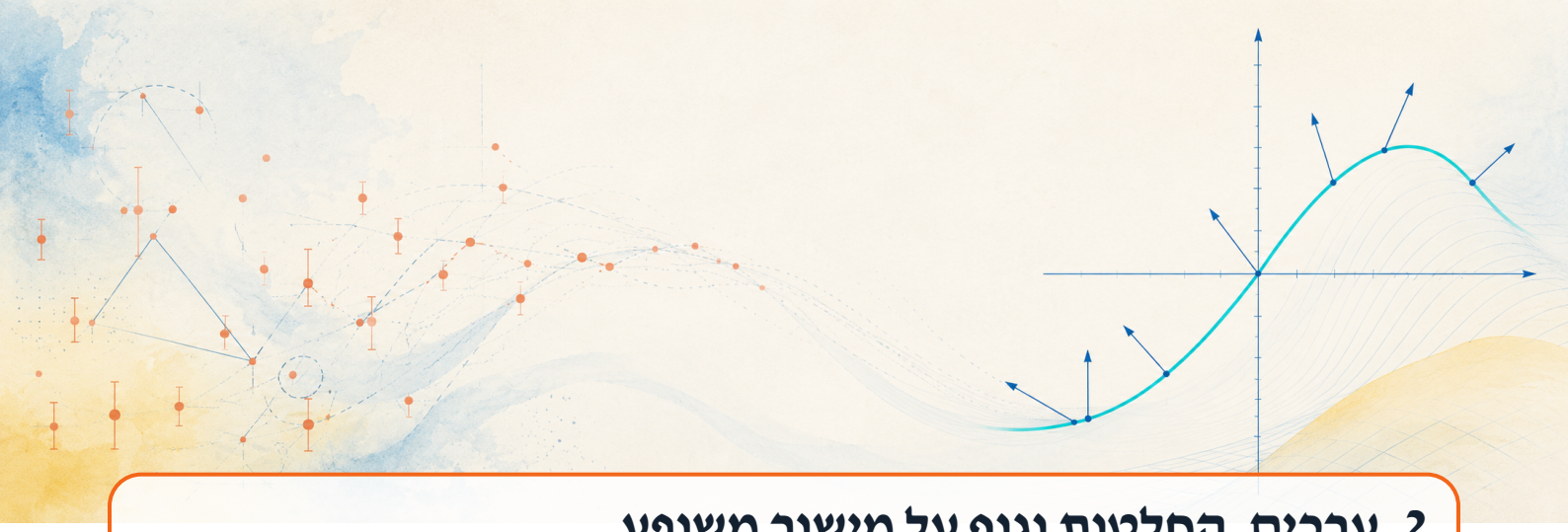
1.8 להעמקה: עקומות ליסאז'ו

בחקירה זו, שהיא רשות ואינה נבחנת, נשתמש במתכון שרטוט מסופק עבור

$$x(t) = A\cos(at + \delta), \qquad y(t) = B\sin(bt).$$

הזמן t בשניות, התדירויות הזוויתיות a, b ברדיאנים לשנייה, הפאזה δ ברדיאנים, ו-A, B הן משרעות הקואורדינטות. העקומה מציגה שתי תנועות סינוסיות בלתי תלויות יחד; אין זה מסלול תחת כוח כבידה מרכזי.

נקבע משרעות ונשווה יחסי תדירות $1:1$, $1:2$ ו-$2:3$. עבור יחס אחד נשנה את הפאזה מ-0 ל-$\pi/2$. נשמור שני גרפים מנוגדים וננסח השערה על סגירות ועל סימטריה. נבדוק אם הארכת תחום הזמן משנה עקומה שנראתה פתוחה: חלון שרטוט סופי אינו הוכחה לאי-סגירות. הפעילות אינה מוסיפה תחביר חובה והיא נפרדת מהבונוס המזכה בנקודות.



2. ערכים, החלטות וגוף על מישור משופע

2.1 הרצאה: ממשתנים לאלגוריתם

בשבוע 1 נתנו שמות למדידות וחישבנו מהן ערכים חדשים. לפני שנוסיף החלטות ולולאות, נאט וננסח במפורש את הרעיון הבסיסי הזה: מהו משתנה, מה עושה פעולת ההצבה וכיצד ערכים משתנים כאשר Julia מבצעת שורה אחר שורה.

2.1.1 משתנה נותן שם לערך

משתנה הוא שם שמפנה לערך. שם מועיל מאפשר לנו לדבר על משמעותו של מספר במקום לכתוב שוב ושוב את המספר עצמו:

להצבה יש צורה כללית name = expression. תחילה Julia מחשבת את הביטוי שמימין, ולאחר מכן השם שמשמאל מפנה לתוצאה. הדוגמה הקטנה ביותר היא $x = 2$. חישוב מדעי עשיר יותר משתמש בכמה שמות בעלי משמעות:

```
mass = 2.0         # kg
speed = 3.0        # m/s
momentum = mass * speed   # kg m/s

println(momentum)   # 6.0
```

```
6.0
```

סימן השוויון היחיד הוא אופרטור ההצבה. תחילה Julia מחשבת את הביטוי שמימין ל-=, ולאחר מכן השם שמשמאל מפנה לערך שהתקבל. לכן בשורה השלישית התוכנית כופלת תחילה את mass ב-speed, ואז מציבה את התוצאה ב-momentum.

תנע של מסה נעה

התנע של גוף מודד כמה תנועה הוא נושא: גוף כבד יותר וגוף מהיר יותר נושאים כל אחד יותר תנועה. עבור מסה נקודתית הנעה הרבה מתחת למהירות האור, $\mathbf{p} = m\mathbf{v}$, ובגודל $p = mv$ לאורך כיוון התנועה. המסה m בקילוגרמים, המהירות v במטר לשנייה, והתנע בקילוגרם-מטר לשנייה ($\mathrm{kg\,m/s}$).

אפשר לבדוק משתנה על ידי כתיבת שמו בתא של מחברת, או להדפיס אותו כחלק מהודעה:

```
momentum
println("momentum = $(momentum) kg m/s")
```

במחברת השורה הראשונה מציגה 6.0; הדוח המודפס הוא:

```
momentum = 6.0 kg m/s
```

2.1.2 התוכנית מתבצעת לפי הסדר

הצבה מבצעת חישוב ברגע שבו מגיעים אליה. היא אינה יוצרת קשר אלגברי קבוע בין משתנים:

```
x = 4
y = x + 1       # y is now 5
x = 10          # x changes, but y is still 5
```

כאשר Julia מבצעת את $y = x + 1$, היא משתמשת בערך הנוכחי של x. שינוי מאוחר יותר של x אינו מחשב מחדש את y; לשם כך צריך לבצע שוב את $y = x + 1$. לסדר השורות חשיבות מיוחדת במחברות, שבהן אפשר להריץ תא יותר מפעם אחת או בסדר שונה.

2.1.3 אפשר להציב במשתנה ערך חדש

אותו שם יכול להפנות לערך חדש בהמשך התוכנית:

```
temperature = 20.0   # C
temperature = temperature + 0.5

println(temperature)     # 20.5
```

```
20.5
```

השורה השנייה אינה משוואה אלגברית. Julia קוראת את הערך הישן בצד ימין, מוסיפה 0.5, ומציבה את התוצאה החדשה בחזרה באותו שם. לולאות ישתמשו בדפוס זה כדי לקדם זמן, להגדיל מונה או לעדכן מקום בסימולציה.

שמות צריכים לבטא משמעות

שמות: שמות משתנים מבחינים בין אותיות גדולות לקטנות, אינם יכולים להכיל רווחים, ויש להשתמש בהם בעקביות. אפשר להשתמש ב-unicode, ובמיוחד באותיות יווניות (למשל להקליד \phi<Tab> כדי לקבל ϕ). נעדיף שמות בעלי משמעות על פני שמות לא מוסברים כגון a, b ו-c. **יחידות**: Julia שומרת בדרך כלל מספרים רגילים בלי היחידות שלהם. בפיסיקה, לעומת זאת, כדאי לעקוב אחריהן. אפשר להוסיף סיומות יחידה לשמות, למשל m_kg, או לחלופין, כפי שאנו עושים כאן, לתעד את היחידה בהערה קצרה בסוף השורה במקום שבו הערך מוגדר, למשל time = 0.35 # s, height = 1.20 # m ו-g = 9.81 # m/s^2; זה אינו חובה אך שימושי מאוד כדי להימנע משגיאות יחידות. במטלת הבונוס 2 (כלומר, שאינה חלק מחומר הקורס הרשמי) נראה שאפשר להשתמש בחבילה Unitful שמוסיפה יחידות למשתנים ומאפשרת לעקוב אחריהן. יש לציין יחידות גם במחרוזות התיעוד או בתיעוד הממשק של פונקציות, ולהבהיר אותן באתרי קריאה שבהם הן עלולות להיות עמומות.

2.1.4 לערכים יש טיפוסים

משתנים יכולים לתת שמות לסוגים שונים של ערכים. בשבוע 1 כבר השתמשנו במספר שלם לספירה, במספרים עשרוניים למדידות ובתוויות טקסט. השבוע נוסיף תוצאות אמת ושקר של השוואות ונציג ארבע הבחנות במפורש, משום שתוכנית שקוראת קלט, בוחרת ענף ומחליטה מתי לעצור חייבת לדעת באיזה סוג ערך היא מטפלת.

טיפוס הוא קטגוריה של ערכים שקובעת כיצד Julia שומרת אותם ואילו פעולות מתאימות להם. למשל, ל-5, ל-0.35, ל-"height" ול-true יש צורות שונות, והם מייצגים מספר שלם, מספר נקודה צפה, טקסט וערך אמת או שקר.

2.1.5 ארבעה טיפוסי ערכים פשוטים

ארבעת הטיפוסים הדרושים כעת

טיפוס ב-Julia	דוגמה	תפקיד מדעי נפוץ
Int	$n = 5$	ספירה ואינדקס של לולאה
Float64	$t = 0.35$ # s	מדידות וגדלים מחושבים
String	label = "height [m]"	קלט טקסטואלי, תוויות והודעות
Bool	ready = true	תשובות אמת/שקר לתנאים

הפונקציה typeof(value) מאפשרת לבדוק את טיפוס הערך. משתמשים בה לאבחון, ולא כתחליף להבנת משמעות הערך.

במחשבי הקורס הרגילים בעלי 64 סיביות, typeof(5) מדפיסה Int64. השם Int ב-Julia מציין את גודל המספר השלם הרגיל של המחשב, ובמחשבים אלה הוא שם חלופי ל-Int64.
הטיפוסים קשורים ישירות לשבוע 1:

```
current = [0.10, 0.20, 0.30, 0.40, 0.50]   # A
n = length(current)                  # Int
I_bar = sum(current) / n             # Float64, A
axis_label = "current I [A]"    # String
analysis_ready = true           # Bool

println(typeof(n))
println(typeof(I_bar))
println(typeof(axis_label))
println(typeof(analysis_ready))
```

מספר שלם ומספר נקודה צפה יכולים להשתתף באותו חישוב. Julia ממירה את המספר השלם לפי הצורך:

```
2 * 0.35       # 0.7, a Float64
5 / 2          # 2.5, a Float64
Float64(5)     # 5.0
```

כללים מעשיים אלה מספיקים כעת. אין צורך בהצהרות טיפוס, בטיפוסים מופשטים או בפרטי הייצוג של מספרי נקודה צפה.

טיפוס אינו יחידה פיסיקלית

ב-t = 0.35 # s, הטיפוס של Julia הוא Float64; היחידה הפיסיקלית היא שניות. היחידה נמצאת בהערה ובתיעוד הממשק הרלוונטי, ולכן ערך אחר מאותו טיפוס Float64 יכול לייצג מטרים או קילוגרמים. בדיקת טיפוסים וניתוח ממדים עונים על שאלות שונות, ואנו זקוקים לשניהם.

2.1.6 השוואות יוצרות ערכי אמת ושקר

תוכנית זקוקה לשאלות שהתשובה עליהן כן או לא כדי לבדוק נתונים ולבחור מה לעשות בהמשך. להשוואה יש צורה left_value operator right_value, והיא מחזירה Bool: הערך true או false. נתחיל בשאלות ישירות שתשובותיהן ברורות:

```
2 < 3          # true
2 == 3         # false
5 != 4         # true
```

אופרטורי ההשוואה העיקריים הם <, =<, >, =>, == (שווה) ו-=! (אינו שווה). סימן שוויון יחיד = מציב ערך; שני סימני שוויון == שואלים שאלה.

אפשר לשלב ערכי אמת ושקר כאשר השוואה אחת אינה מספיקה. האופרטורים &&, || ו-! פירושם וגם, או, ולא:

```
true && false      # false: both must be true
true || false      # true: at least one is true
!true              # false
```

אותן צורות יכולות לבטא כלל פיסיקלי עשיר יותר לבדיקת תקינות:

```
t = 0.35   # s
z0 = 1.20   # m
nonnegative_time = t >= 0
valid_height = z0 >= 0
valid_input = nonnegative_time && valid_height
```

2.1.7 החלטה ראשונה בעזרת if

חישוב ערך אמת או שקר מועיל רק אם התוכנית יכולה לפעול לפיו. הוראת if היא תנאי: היא מריצה בלוק אחד כאשר התנאי true. בלוק else אופציונלי רץ כאשר התנאי false. הצורה הכללית היא:

```
if condition
    # commands when the condition is true
else
    # commands when the condition is false
end
```

כל תנאי מסתיים ב-end. אם אין צורך לעשות דבר במקרה השקרי, משמיטים את בלוק ה-else. הנה הדוגמה הדו-ענפית הקטנה ביותר:

```
temperature = 25.0   # C
if temperature > 30
    println("hot")
else
    println("not hot")
end
```

```
not hot
```

לעתים המשך החישוב ייצור תוצאה חסרת משמעות. Julia מספקת עצירה מכוונת בצורה זו:

```
throw(ArgumentError("message"))
```

היא עוצרת את הביצוע ומדווחת מדוע. נשתמש בה לאחר המרה שנכשלה ולאחר קלט שאינו חוקי מבחינה פיסיקלית.

2.2 קינמטיקה בחלקים כהחלטה

תאוצה מסומנת לעומת גודל הכבידה החיובי

גובה חיובי כלפי מעלה, ולכן תאוצת הכובד היא

$$g = -9.81\ \mathrm{m/s^2}.$$

עבור גובה התחלתי z_0 ומהירות התחלתית v_0,

$$z_{\mathrm{free}}(t) = z_0 + v_0 t + \frac{1}{2} g t^2.$$

לשלושת האיברים יחידות של מטרים. בדיקת ממדים זו אינה תלויה בכך שערכי הקוד הם מטיפוס Float64.

המודל ללא קפיצה אינו מאפשר לכדור להמשיך מתחת לקרקע:

$$z(t) = \begin{cases} z_{\text{free}}(t), & z_{\text{free}}(t) > 0, \\ 0, & z_{\text{free}}(t) \le 0. \end{cases}$$

ההגדרה בחלקים מתורגמת ישירות לתנאי:

```
t = 0.35   # s
z0 = 1.20   # m
v0 = 0.0    # m/s
g = -9.81   # m/s^2

if t < 0 || z0 < 0
    throw(ArgumentError("time and initial height must be non-negative"))
end

z_free = z0 + v0 * t + 0.5 * g * t^2   # m

if z_free > 0
    z = z_free
    state = "in air"
else
    z = 0.0
    state = "on the ground"
end

println("at t = $(t) s: z = $(round(z, digits=3)) m ($(state))")
```

```
at t = 0.35 s: z = 0.599 m (in air)
```

ב-$t = 0.35\ \text{s}$ התוכנית מדווחת בקירוב $z = 0.599\ \text{m}$. התוצאה עוברת שלוש בדיקות: הזמן בתחום המודל, הגובה נמוך מן הגובה ההתחלתי, ולכל איבר במשוואת המקום יחידות של מטרים.

2.2.1 כמה ענפים

משתמשים ב-elseif כאשר יש יותר משני מקרים זרים זה לזה שיש לבדוק לפי סדר. הצורה הכללית היא if, אחריו מספר כלשהו של ענפי elseif, ענף else אופציונלי, ולבסוף end אחד. רק הענף האמיתי הראשון רץ. דוגמה קטנה לסיווג סימן מציגה את המבנה:

```
value = -2
if value < 0
    description = "negative"
elseif value == 0
    description = "zero"
else
    description = "positive"
end
```

אותו מבנה נותן סיווג שימושי יותר של גובה:

```
if z == 0
    state = "on the ground"
elseif z < 0.20
    state = "close to the ground"
else
    state = "in air"
end
```

לעתים דרושות החלטות מקוננות, אך שרשרת שטוחה של else/elseif/if קלה יותר לבדיקה כאשר היא מבטאת אותו היגיון.

שלוש קטגוריות של שגיאות

- **שגיאת תחביר:** Julia אינה יכולה לפענח את התוכנית, למשל בגלל end או סוגר חסרים.
- **שגיאת זמן ריצה:** התוכנית מתחילה ואז מגיעה לפעולה לא חוקית, למשל העלאת String בחזקה.
- **שגיאה לוגית:** התוכנית רצה אך ממשת אלגוריתם או פיסיקה שגויים, למשל g חיובי כאשר גובה חיובי כלפי מעלה, או זמן שאינו מתקדם בלולאת while.

קוראים את השגיאה המדויקת ואת מספר השורה, מבודדים את הבלוק הקטן ביותר שנכשל, בודקים ערכים וטיפוסים, בודקים ממדים וסימנים, ומריצים מקרים שתשובתם ידועה.

כעת נגרום לתוכניות לבחור בין מקרים פיסיקליים נבדלים. לפני שאפשר לסמוך על חישוב, עלינו לדעת איזה סוג של ערך אנו מחזיקים, להמיר במכוון, ולהצהיר על התחום שבו מודל תקף.

2.3 לערכים יש טיפוסים

ערכים ב-Julia נושאים טיפוס. מספרים שלמים, מספרי נקודה צפה, טקסט וערכי אמת משמשים למטרות שונות.

```
println(3, "  ->  ", typeof(3))
println(3.0, "  ->  ", typeof(3.0))
println("3", "  ->  ", typeof("3"))
println(3 < 4, "  ->  ", typeof(3 < 4))
```

```
3  ->  Int64
3.0  ->  Float64
3  ->  String
true  ->  Bool
```

הטיפוס השלם המדויק יכול להיות תלוי במחשב; ההבחנה החשובה היא בין Int, נקודה צפה, String ו-Bool.

מספר שלם ומספר נקודה צפה יכולים להתערבב באריתמטיקה רגילה; Julia מקדמת את התוצאה לפי הצורך. טקסט אינו הופך בשקט למספר. אנו בוחרים המרות במפורש בשני הכיוונים.

```
println(2 + 0.5)
text = "12.5"
mass = parse(Float64, text)   # kg
println(mass + 0.5)
println(string(mass, " kg"))
println(Int(7.0))
```

```
2.5
13.0
12.5 kg
7
```

parse(Float64, "twelve") מעלה שגיאה משום שהטקסט אינו ספרה של נקודה צפה. גם Int(7.2) מעלה שגיאה: המרה ל-Int מותרת רק כאשר הערך שלם בדיוק. זה מגן עלינו מפני השלכת מידע בשקט.

2.4 בוליאנים והחלטות הדרגתיות

השוואות מפיקות ערכי Bool. נשאל תחילה שאלה אחת, אחר כך נבחר בין שני מקרים, ולבסוף נוסיף מקרה שלישי עם elseif.

```
temperature = -2.0   # C
println(temperature < 0)
if temperature < 0
    println("below freezing")
else
    println("not below freezing")
end
```

```
true
below freezing
```

```
if temperature < 0
    state = "ice"
elseif temperature == 0
    state = "melting-point"
else
    state = "liquid"
end
println(state)
```

```
ice
```

ice. בדיוק ענף אחד משרשרת else/elseif/if רץ. = מציב; == משווה.

אנו משלבים תנאים עם && (שניהם), || (לפחות אחד) ו-! (לא). בדיקת תקינות שואלת לעתים קרובות אם ערך הוא מספר סופי. isfinite(value) מחזירה true עבור מספרים סופיים רגילים ו-false עבור NaN, Inf או Inf-, שאינם מדידות שמישות. אפשר לשלב בדיקה זו עם תנאי פיסיקלי:

```
is_finite_positive = isfinite(2.0) && 2.0 > 0
println(is_finite_positive)
println(!is_finite_positive || 2.0 == 2)
```

```
true
true
```

2.5 זוויות, לוגיקה ואימות לפני חישוב

תנאי הוא שאלה מתמטית מדויקת

a && b אמיתי רק כאשר שני התנאים אמיתיים; a || b אמיתי כאשר לפחות אחד אמיתי. xor(a,b) אמיתי כאשר בדיוק אחד אמיתי. הכללים מסייעים לבחירת מקרים זרים, אך המודל חייב לקבוע בנפרד מה קורה בשוויון. אי-שוויון משורשר כמו $0 \le \theta < 90^\circ$ דורש ששני הגבולות יתקיימו; אפשר לכתוב שתי השוואות המחוברות ב-&&.

לזווית דרושה יחידה מפורשת. sin ו-cos מצפות לרדיאנים, ואילו sind ו-cosd מצפות למעלות; rad2deg ממירה זווית מרדיאנים למעלות. הצורה הדו-ארגומנטית atan(y, x) מחזירה את הזווית של הנקודה (x, y) תוך התחשבות ברביע: היא משתמשת בשני הסימנים, ולא רק ביחס ביניהם, ולכן יכולה להבחין בין כיוונים מנוגדים. דוגמה קטנה לכל אחת מהן היא:

```
println(true && false)
println(true || false)
println(xor(true, true))
println(xor(true, false))
println(sin(pi / 6))
```

```
println(sind(30.0))
println(round(rad2deg(atan(1.0, -1.0)); digits=1))
```

```
false
true
false
true
0.49999999999999994
0.5
135.0
```

sin מקבלת רדיאנים ו-sind מקבלת מעלות. `atan(y,x)` משתמשת בשני הסימנים כדי לבחור רביע נכון: הנקודה $(-1, 1)$ נמצאת בזווית $135°$, לא $-45°$. בראשית אין כיוון פיסיקלי מוגדר, ולכן יישום חייב לקבוע מוסכמה. הבדלים זעירים בין חישובי נקודה צפה שקולים אינם הבדל בפיסיקה.

המרת טקסט ואימות פיסיקלי הם שלבים נפרדים

readline() מחזירה טקסט. `parse(Float64, text)` הופכת טקסט מספרי תקין למספר; טקסט פגום גורם לשגיאה. המרה מוצלחת אינה מספיקה: NaN ו-Inf אינם מדידות סופיות, וגם מסה שלילית סופית נמצאת מחוץ למודל. נאמת לפני שורש, חלוקה או בנייה טריגונומטרית. `throw(ArgumentError("message"))` עוצרת את החישוב; הדפסת הודעה בלבד אינה עוצרת אותו.

לפני בדיקת תחום פיסיקלי, אנו בודקים שהערך סופי: isfinite(value) הוא true רק עבור מספרים סופיים רגילים ו-false עבור NaN או Inf. המקטע הבא ממיר טקסט ומאמת את התוצאה כמסה:

```
text = "2.5"  # reproducible example of text supplied by readline()
mass = parse(Float64, text)   # kg
if !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite and positive"))
end
println("mass = ", mass, " kg")
```

```
mass = 2.5 kg
```

הדוגמאות הקודמות של tryparse ו-nothing נשמרות לקריאת רשות על טיפול בקלט. מסלול החובה השבוע משתמש ב-parse ובבדיקות תחום מפורשות; אין צורך בהגדרת פונקציה באף אחד מהשלבים.

2.6 מודל חיכוך בחלקים על מישור משופע

כוחות, סימנים והנחות

נבחר $+x$ במעלה המישור הקבוע ו-$+y$ ניצב לו. עבור גוף בעל מסה m (ק"ג), זווית θ (רדיאנים ב-cos/sin) וגודל כבידה חיובי g ($\mathrm{m/s^2}$), לרכיב הכבידה במורד המישור יש גודל $mg\sin\theta$ והכוח הנורמלי הוא $N = mg\cos\theta$. החיכוך הסטטי יכול להתנגד לתנועה הממשמשת במורד עד $f_{s,\max} = \mu_s N$. אם אינו מצליח להחזיק, החיכוך הקינטי $f_k = \mu_k N$ פועל במעלה המישור ו-$a = (-mg\sin\theta + \mu_k N)/m$. אנו מניחים חיכוך קולון, מגע קשיח, ללא גרר, ו-$0 \le \theta < 90°$. המודל אינו מתאר גלגול, מגע משתנה או חיכוך תלוי מהירות. ב-$\theta = 0$ החיכוך הסטטי הנדרש הוא אפס; אם $\mu_k = 0$, ההחלקה נותנת $a = -g\sin\theta$.

אנו מאמתים לפני שימוש בקלט. throw(ArgumentError(...)) עוצר מיד את החישוב ומודיע לקורא שהתנאי המתועד לקלט לא התקיים. זה שונה מ-println, שמדפיס טקסט בלבד וממשיך. בדיקת השייכות value isa T שואלת אם ערך שייך לטיפוס T; Real הוא שם המשפחה של טיפוסי המספרים הממשיים ב-Julia, הכוללת מספרים שלמים וערכי נקודה צפה רגילים. Bool דורש זהירות מיוחדת כאן: משום ש-true isa Real גם הוא true, בדיקה ל-Real בלבד תקבל true כמסה, ולכן נדחה אותו במפורש.

```
mass = 2.0      # kg
angle = 30.0    # deg
 μ_s = 0.60
 μ_k = 0.40
g = 9.81        # m/s^2
if !(mass isa Real) || mass isa Bool || !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite, positive, and non-Boolean
"))
elseif !isfinite(angle) || !(0 <= angle < 90) || μ_k < 0 || μ_k > μ_s
    throw(ArgumentError("use 0 <= angle < 90 and 0 <= μ_k <= μ_s"))
end
 θ = deg2rad(angle)   # rad
normal = mass * g * cosθ()   # N
needed_static = mass * g * sinθ()   # N
if needed_static <= μ_s * normal
    state = "stuck"
    acceleration = 0.0   # m/s^2
else
    state = "sliding"
    acceleration = (-needed_static + μ_k * normal) / mass   # m/s^2
end
println(state, ", a = ", acceleration, " m/s^2")
```

```
stuck, a = 0.0 m/s^2
```

stuck, a = 0.0 m/s^2. כאן $mg\sin 30^\circ = 9.81$ N ו-$\mu_s N \approx 10.19$ N, ולכן החיכוך הסטטי מספיק. אם נשנה רק את μ_s ל-0.40, התוצאה תהיה sliding עם תאוצה שלילית: במורד המישור לפי המוסכמה שהצהרנו. שוויון שייך לענף ה-stuck, מוסכמת הסף המפורשת שלנו.

2.7 תרגול: משיכה יכולה להפוך את כיוון התנועה הצפוי

נוסיף למודל המישור כוח משיכה ידוע P כלפי מעלה. לפני בחירת סימן החיכוך נחשב את הנטייה ללא חיכוך, $T = P - mg\sin\theta$. כאשר T חיובי, התנועה הצפויה היא במעלה המישור; כאשר הוא שלילי, התנועה הצפויה היא במורד. החיכוך מתנגד לנטייה זו, וחיכוך סטטי יכול להחזיק את הגוף כאשר $|T| \le \mu_s N$.

נגיע למפגש כשאנו יודעים לקרוא שרשרת תנאים ולהמיר מעלות לרדיאנים. בחלק המודרך הקצר נחזה כיוון, נעקוב אחר ענף אחד ונבדוק את $T = 0$. לאחר מכן נתחיל את מטלת מעבר הקיר. נתרגל שיטה חוזרת: נאמת קלט, ננסח תנאי פיסיקלי, נחשב את המקרה המתאים ונבדוק את גבולו.

2.8 מטלת החובה: האם קליע עובר מעל קיר?

נשגר מגובה הקרקע במהירות v_0 ובזווית θ, עם גודל כבידה $g > 0$. נגדיר $+x$ לכיוון הקיר ו-$+y$ מעלה. במודל של כבידה אחידה ללא גרר,

$$v_x = v_0\cos\theta, \quad t_w = x_w/v_x, \quad y_w = v_0\sin\theta\, t_w - \tfrac{1}{2}gt_w^2, \quad v_{y,w} = v_0\sin\theta - gt_w.$$

נדרוש $v_x > 0$ לפני החלוקה. נשווה את y_w לגובה הקיר ונשתמש ב-$v_{y,w}$ לסיווג עלייה או ירידה. שוויון בגובה נחשב מעבר; מהירות אנכית אפס שייכת לקטגוריית הירידה לפי המוסכמה שפורסמה. גובה התנועה החופשית המחושב עשוי להיות שלילי: עבור קיר שגובהו אינו שלילי, פירושו אי-מעבר ולא המשך פיסיקלי דרך הקרקע.

נכתוב תחזית למקרה רגיל ולמקרה גבול. תרגיל בית 2 הוא תוכנית שלמה המוגשת ב־CodeRunner: היא מדפיסה חמש בקשות קלט, קוראת את הערכים באמצעות readline(), מאמתת את התחום הפיסיקלי, ומדפיסה שורת תוצאה אחת כאשר הקליע נוחת לפני הקיר או שלוש שורות אחרת. ה־CodeRunner מספק חמישה ערכי קלט ומשווה את הפלט השלם, כולל בקשות הקלט. נעגל את הגובה המוצג לארבע ספרות

אחרי הנקודה, ונשתמש בערך המלא לפני עיגול כדי לקבוע את המצב ואת התוצאה. החלטת המעבר וסיווג התנועה נשארים נפרדים: אפשר לעבור את הקיר גם בירידה. אין צורך בלולאות או בהגדרת פונקציות.

2.9 בונוס לבחירה: יחידות ופאזורים בזרם חילופין

הבונוס הוא רשות, אינו תנאי להמשך ואינו נבחן. הוא מזכה ב**שליש נקודה בציון הסופי**.

הקריאה העצמאית מפתחת מספרים מרוכבים, גודל, פאזה, צמוד וקשר אוילר לפני השימוש ב-cis. נבחר משרעות שיא ואת מוסכמת הזמן $\exp(+i\omega t)$, נקבע את פאזת המתח לאפס, נבנה $Z = |Z|\exp(i\phi_Z)$ ונחשב $I = V/Z$. הזרם המדיד הוא החלק הממשי של $I\exp(i\omega t)$; המספר המרוכב הוא ייצוג ולא זרם נמדד נוסף.

נעקוב אחר דוגמאות Unitful, נוודא שלמתח חלקי התנגדות יש יחידות זרם ונבדוק נגד בעל פאזה אפס לפני מקרה של רבע מחזור. נגיש קטע המציב זרם מרוכב באמפר ונריץ את בדיקות היחידות השקולות והקלט הפסול. נסביר מדוע פאזה חיובית של העכבה נותנת פאזה שלילית של הזרם. המודל לינארי, בעל תדירות אחת ובמצב מתמיד; מעברי מיתוג ורכיבים לא-לינאריים אינם מתוארים בו.

2.10 להעמקה: החזרה פנימית מלאה

נשתמש במתכון המסופק לחוק סנל $n_1\sin\theta_1 = n_2\sin\theta_2$, עם מקדמי שבירה חיוביים וזוויות הנמדדות מהנורמל לממשק. עבור $n_1 > n_2$, זווית פגיעה הגדולה מהזווית הקריטית $\theta_c = \arcsin(n_2/n_1)$ גורמת להחזרה פנימית מלאה. נסווג דוגמאות מתחת לסף, על הסף ומעליו לפני חישוב סינוס הפוך; הפעלת asin על מספר גדול מאחת ללא בדיקה אינה חישוב פיסיקלי תקין.

נשרטט החזרות אידיאליות חוזרות בסיב ישר ונשנה את זווית הכניסה. נרשום אילו בחירות נשארות מונחות בסיב ונסביר מהו הנורמל בכל ממשק. לולאת עקיבה מסופקת, אם נשתמש בה, היא הצצה לבחירה ולא חומר חובה בשבוע 2. אופני גל, נפיצה, הפסדי כיפוף ובליעה דורשים פיסיקה נוספת. החקירה נפרדת מבונוס הפאזורים, אינה נבחנת ואינה תנאי להמשך.



3. לולאות, וקטורים ותנועה נדגמת

שאלות פיסיקליות רבות דורשות אותו חישוב בזמנים או במקומות רבים. כעת ננסח את החזרה במפורש ונשמור את ההיסטוריה שלה בווקטורים. וקטור הוא אוסף מסודר של ערכים; לולאה חוזרת על גוש הוראות; ביחד הן נותנות לנו דרך לצעוד עם מודל קדימה בזמן. מטרתנו היא להבין אילו הוראות חוזרות, מה משתנה בכל צעד ואיזו ראיה קובעת מתי נעצור. נקרא לכלים קיימים; הגדרת פונקציות משלנו שייכת לשבוע 5.

3.1 וקטורים שומרים היסטוריה מסודרת

3.1.1 יצירת וקטורים

וקטור הוא אוסף מסודר, הכתוב בסוגריים מרובעים. אפשר לכתוב וקטור ישירות, להתחיל וקטור ריק, או לבקש וקטור מוכן של אפסים או אחדות:

```
v = [10, 20, 30, 40]
empty_int = Int[]
println(v)
println(empty_int)
println(zeros(5))
println(ones(4))
println(collect(1:5))
println(range(0, 1; length=6))
```

```
[10, 20, 30, 40]
Int64[]
[0.0, 0.0, 0.0, 0.0, 0.0]
[1.0, 1.0, 1.0, 1.0]
[1, 2, 3, 4, 5]
0.0:0.2:1.0
```

הביטוי `1:5` הוא טווח (range), לא וקטור: הוא מתאר את הערכים 1 עד 5 בלי לאחסן אותם. נקרא ל-collect כאשר אנו זקוקים לווקטור ממשי. הטווח `0:0.2:1.0` שנוצר על ידי `range(0, 1; length=6)` מופיע כטווח כי שש נקודות המרווחות שווה מ-0 עד 1 רחוקות 0.2 זו מזו. `zeros(5)` נותן וקטור של חמישה אפסים בנקודה צפה; `ones(4)` נותן ארבעה אחדות. `Int[]` הוא וקטור ריק שאיבריו יהיו שלמים. מערכי Julia בני שינוי: אפשר לשנות איברים או אורך אחרי היצירה, כפי שנראה מיד.

3.1.2 אינדוקס ופרוסות

הווקטורים הרגילים של Julia מתחילים באינדקס 1. אינדקס מציין מקום ולא את הערך שבו; end בתוך ביטוי אינדוקס מציין את המקום האחרון.

```
v = [10, 20, 30, 40, 50]
println(v[1])
println(v[end])
println(v[2:4])
println(v[2:end])
v[2:3] = [99, 88]
println(v[[1, 3, 5]])
indices = [2, 3]
println(v[indices])
println(v)
```

```
10
50
[20, 30, 40]
[20, 30, 40, 50]
[10, 88, 50]
[99, 88]
[10, 99, 88, 40, 50]
```

פרוסה כמו `v[2:4]` כוללת את שני הקצוות. אפשר להשיב לפרוסה, וניתן לבחור כמה מקומות בבת אחת על ידי העברת וקטור של מקומות לאינדקס. הפונקציות `length(v)`, `firstindex(v)` ו-`lastindex(v)` מדווחות על מספר האיברים ועל המקום הראשון והאחרון התקף.

פרוסה היא העתק; שם חלופי הוא אותו וקטור

עבור וקטור רגיל, `part = height[2:3]` מעתיק את האיברים ההם. שינוי `part[1]` אינו משנה את `height`. לעומת זאת, `same = height` יוצר שם שני המתייחס לאותו וקטור; שינוי `same[1]` משנה את האובייקט המשותף. `copy(a)` יוצר העתק אמיתי ובלתי תלוי. תצוגה (view) חולקת את האחסון עם המקור בלי להעתיק: `@view a[1:2]` מאפשר לקרוא ולכתוב את שני האיברים הראשונים של `a` דרך שם חדש. הסימן `@` מסמן מאקרו, טרנספורמציה מובנית קטנה של הביטוי שאחריו; אנו משתמשים בו כאן רק כדי לבקש את התצוגה, ואיננו צריכים לכתוב מאקרוים בעצמנו.

```
a = [1, 2, 3]
b = a
c = a[1:2]
d = copy(a)
e = @view a[1:2]
a[1] = 2
println("a[1]=", a[1], " b[1]=", b[1], " c[1]=", c[1], " d[1]=", d[1], "
 e[1]=", e[1])
```

```
a[1]=2 b[1]=2 c[1]=1 d[1]=1 e[1]=2
```

השם החלופי `b` והתצוגה `e` עוקבים אחרי השינוי; ההעתק `d` והפרוסה `c` אינם עוקבים. איזו התנהגות אנו רוצים תלוי בכך אם הכוונה היא לאותו אחסון או לתמונת מצב שלו.

3.1.3 שינוי וקטור

סימן קריאה `!` בסוף שם פונקציה הוא המוסכמה של Julia לפעולה משנה: היא עשויה לשנות את הטיעון עצמו. `push!` מוסיף ערך אחד, `append!` מוסיף כל איבר של אוסף, ו-`pop!` מסיר ומחזיר את האיבר האחרון. `insert!` ו-`deleteat!` משנים מקום נבחר.

```
v = [1, 2, 3]
push!(v, 4); println(v)
```

```
append!(v, [5, 6]); println(v)
pop!(v); println(v)
insert!(v, 2, 99); println(v)
deleteat!(v, 2); println(v)
w = vcat(v, [7, 8]); println(w); println(v)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5]
[1, 99, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 7, 8]
[1, 2, 3, 4, 5]
```

שימו לב להבדל: `push!` מוסיף עצם אחד, `append!` מוסיף כל איבר של אוסף, ו-`vcat` בונה וקטור מחובר חדש ומשאיר את `v` ללא שינוי. בדוגמאות הלולאה הבאות נגדיל היסטוריות בעזרת `push!`, שהיא הדרך הטבעית לאחסן רצף של דגימות תוך חישובן.

3.1.4 חשבון וקטורים ופונקציות שימושיות

חשבון על וקטורים פועל איבר אחרי איבר, ופונקציות אחדות מסכמות וקטור שלם:

```
a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]
println(a + b)
println(2a)
println(sum(a)); println(maximum(a)); println(minimum(a)); println(
length(a))
println(a .* b)
println(a .^ 2)
println(sqrt.(a))
using LinearAlgebra
println(dot(a, b))
println(round(norm([1,1]); digits=4))
```

```
[5.0, 7.0, 9.0]
[2.0, 4.0, 6.0]
6.0
3.0
1.0
3
[4.0, 10.0, 18.0]
[1.0, 4.0, 9.0]
[1.0, 1.4142135623730951, 1.7320508075688772]
32.0
1.4142
```

הנקודה מסמנת פעולה איבר-איבר: `a .* b` מכפיל איברים תואמים, בעוד `a * b` היה שגיאה עבור צורות אלה. אופרטורים מנוקדים פועלים איבר-איבר, כך ש-`a .^ 2` מעלה בריבוע כל איבר ו-`sqrt.(a)` מפעיל שורש ריבועי על כל איבר. `sum`, `maximum`, `minimum` ו-`length` מדווחים מספר אחד עבור הווקטור כולו. המודול התקני `LinearAlgebra` מספק את המכפלה הסקלרית `dot(a, b)` ואת האורך `norm(a)`; כמו `Statistics`, הוא חלק מ-Julia ואין צורך בהתקנה.

תרגיל מהיר

צרו וקטור של חמישה ערכים, החליפו את איברו השלישי, הוסיפו שני ערכים נוספים, וחשבו את הממוצע שלו באמצעות `sum(v) / length(v)`. לאחר מכן בדקו את הממוצע עם `using Statistics; mean(v)`.

3.2 חזרה מתוכננת: לולאת for

לולאה חוזרת על גוש הוראות. לולאת for עוברת לפי הסדר על אוסף ערכים מסופק: `for value in collection`, גוף הלולאה ולבסוף end. לפני ההרצה נרשום אילו ערכים יקבל משתנה הלולאה. הערה מתחילה ב-#; היא מסבירה בחירה לקורא ואינה מתבצעת.

```
for n in 1:3
    println(n)
end
```

```
1
2
3
```

הטווח `1:3` כולל את שני קצותיו. הצורה `start:step:stop` מתקדמת בצעד שנקבע בלי לעבור את הגבול; צעד שלילי מאפשר ספירה לאחור. הקצה נכלל רק כאשר סדרת הצעדים מגיעה אליו.

```
for time in 0:2:10   # s
    println("time = $time s")
end
```

```
time = 0 s
time = 2 s
time = 4 s
time = 6 s
time = 8 s
time = 10 s
```

סימן הדולר מכניס ערך למחרוזת. עבור חישוב ולא שם יחיד נשתמש ב-`$(expression)`, למשל `println("square = $(n^2)")`.

יש כמה דרכים לעבור על איברי וקטור. משתנה הלולאה יכול לקבל ערכים שלמים, את הערכים עצמם, אינדקסים תקפים, או זוג אינדקס-ערך:

```
v = [10, 20, 30]
for i in 1:3
    println(i)
end
for x in v
    println(x)
end
for i in eachindex(v)
    println("index = ", i, ", value = ", v[i])
end
for (i, x) in enumerate(v)
    println(i, " => ", x)
end
```

```
1
2
3
```

```
10
20
30
index = 1, value = 10
index = 2, value = 20
index = 3, value = 30
1 => 10
2 => 20
3 => 30
```

איזו צורה נבחר תלוי במה שהגוף צריך. כאשר רק ערכים חשובים, `for x in v` הוא הבהיר ביותר. כאשר אנו עושים אינדוקס או משנים איברים, `eachindex(v)` מספק את האינדקסים התקפים עבור אותו אוסף, כך שאיננו צריכים להניח טווח אינדקסים מסוים. `enumerate` מספק מונה לצד כל איבר; המונה שלו הוא המקום הרגיל המתחיל ב-1 כאן, וזה כל מה שאנו צריכים.

משתנה צובר מעביר תוצאה מחזרה לחזרה

כדי לחבר ערכים נאתחל סכום, נוסיף תרומה אחת ונשמור את הסכום החדש לצעד הבא. האתחול חייב להיות מחוץ לגוף החוזר; איפוס בתוכו מוחק את התרומות הקודמות. מונה מתעד כמה צעדים ביצענו, ואילו משתנה צובר מתעד את תוצאתם המצטברת. הדוגמאות שלהלן מקבצות את ההצהרות שלהן בגוש `begin ... end`, כך שכל דוגמה נקראת כיחידה אחת וניתן להדביק ולהריץ אותה כמו שהיא.

```
begin v = [3, 5, 8, 2]
    total = 0
    for x in v
        total += x
    end
    println(total)
    println(sum(v))
end
```

```
18
18
```

הלולאה משחזרת את `sum(v)` חיבור אחד בכל פעם. זהו השלד של כמעט כל צבירה בקורס הזה: להתחיל מערך נייטרלי, לשלב איבר אחד בכל חזרה, ולשמור את התוצאה לצעד הבא. עצרת היא אותה תבנית עם כפל במקום חיבור:

```
begin f = 1
    for n in 1:5
        f = f * n
    end
    println(f)
end
```

```
120
```

מונה הוא צובר של צעדים המקיימים תנאי:

```
begin v = [-2, 3, 0, 7, -1]
    count_positive = 0
    for x in v
        if x > 0
            count_positive += 1
        end
```

```
    end
    println(count_positive)
end
```

```
2
```

מציאת מקסימום בלי `maximum` משתמשת בצובר שרק משתפר:

```
begin v = [4, 9, 2, 7]
    largest = v[1]
    for x in v
        if x > largest
            largest = x
        end
    end
    println(largest)
end
```

```
9
```

בניית וקטור של סכומים רצים משלבת צובר עם `push!`:

```
begin v = [2, 3, 5, 7]
    running = 0
    cumulative = Int[]
    for x in v
        running += x
        push!(cumulative, running)
    end
    println(cumulative)
    println(cumsum(v))
end
```

```
[2, 5, 10, 17]
[2, 5, 10, 17]
```

כאן הווקטור `cumulative` מתעד את המצב בכל צעד, ולא רק את האחרון. זו התבנית שנשתמש בה כדי לאחסן מסלול: לדחוף כל דגימה חדשה תוך חישובה, ולהשוות לתוצאה ידועה.

חישוב וקטורי נפוץ הוא המכפלה הסקלרית, שגם היא משתמשת בצובר על פני אינדקסים תואמים:

```
using LinearAlgebra
begin
    a = [1.0, 2.0, 3.0]
    b = [4.0, 5.0, 6.0]
    s = 0.0
    for i in eachindex(a, b)
        s += a[i] * b[i]
    end
    println(s)              # 32.0
    println(dot(a, b), " ", ·ab)   # 32.0 32.0
end
```

```
32.0
32.0 32.0
```

שני הווקטורים חייבים להיות באותו אורך כדי שהלולאה תהיה הגיונית; זה בדיוק מה ש-`eachindex(a, b)` דורש. צבירת סכום עם `s += a[i] * b[i]` היא הגרסה הידנית של המכפלה הסקלרית.

3.3 סיכומים: ממוצע וסטיית תקן מדגמית

מיקום ופיזור עונים על שאלות שונות

עבור n תצפיות q_i, הממוצע הוא $\bar{q} = \sum_i q_i / n$. סטיית התקן המדגמית המתוקנת היא

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(q_i - \bar{q})^2}, \qquad n \geq 2.$$

לשניהם יחידות של q. הממוצע מאתר את המדגם; סטיית התקן מודדת את פיזורו. לא נקודה עשרונית קטנה יותר ולא פיזור קטן שוללים שגיאת מדידה שיטתית. פרשנות של $s/\sqrt{n}$ כאיז וודאות של ממוצע דורשת הנחות נוספות על מדידות חוזרות בלתי תלויות.

`Statistics` הוא מודול מספריה תקני המסופק עם Julia, כך שאין צורך בהתקנת חבילה. ההצהרה `using Statistics` מנגישה את הפונקציות `mean` ו-`std`:

```
using Statistics
measurements = [1.0, 2.0, 3.0]   # m
println(mean(measurements))
println(std(measurements))
```

```
2.0
1.0
```

ברירת המחדל של `std` משתמשת בתיקון שלעיל, ושתי הפונקציות פועלות על וקטור שלם. מסלול אינו אוטומטית אוסף של מדידות חוזרות בלתי תלויות; פיזור של מקומות אינו מעריך בעצמו איז וודאות אינסטרומנטלית.

3.4 חזרה המותנית בתנאי: while

לולאת `while` בודקת את התנאי שלה לפני כל איטרציה. נזהה את האתחול, התנאי, העדכון והנימוק לעצירה שלה. היא עשויה להתבצע אפס פעמים. `break` עוזב את הלולאה הקרובה ביותר מיד; בחיפוש חסום היא מאפשרת לנו לעצור בהצלחה הראשונה.

```
for n in 1:10
    if n^2 >= 20
        println("first square at least 20 is $(n^2)")
        break
    end
end
```

```
first square at least 20 is 25
```

העצירה היא חלק מהמודל המדעי

אנו מבחינים בין הדגימה המוצלחת הראשונה לבין הזמן המדויק שבו מתרחש אירוע רציף. אנו שומרים חסם של מספר צעדים מרבי כאשר העצירה לא הוכחה. עדכון חסר יכול ליצור לולאה אינסופית; אי-שוויון שגוי יכול לדלג על אירוע התחלתי או גבולי. תוכניות קלט-שומר-סף ותרגילי `continue` מתקדמים הם קריאה אופציונלית, לא תנאי מוקדם כאן.

3.5 מהנגזרת לעדכון אוילר

תנועת קליע: חוקים ומגבלות

אנו לוקחים $+x$ ימינה ו-$+y$ מעלה. קליע נקודתי בכבידה אחידה בקירוב, בלי התנגדות אוויר, מקיים $\dot{x} = v_x$, $\dot{y} = v_y$, $\dot{v}_x = 0$, ו-$\dot{v}_y = -g$. מקומות במטרים, מהירויות ב-m/s, זמן בשניות, ו-$g > 0$ ב-$\mathrm{m/s}^2$. המהירות האופקית חייבת להישאר קבועה. התנגדות, הקפצה, שינוי כבידה ועקמומיות כדור הארץ מושמטים; צעד מספרי אינו יכול להחזיר את ההשפעות החסרות.

ערכים ישנים קובעים את המצב הבא

הגדרת הנגזרת נותנת $[y(t + \Delta t) - y(t)]/\Delta t \approx v_y(t)$ על פני צעד קטן. לפיכך אוילר מפורש משתמש ב-

$$x_{n+1} = x_n + v_{x,n}\Delta t, \quad y_{n+1} = y_n + v_{y,n}\Delta t, \quad v_{y,n+1} = v_{y,n} - g\Delta t.$$

עדכון המקום עם המהירות החדשה במקום נותן אלגוריתם שונה. הגובה המדויק בדוגמת הכבידה הקבועה הזאת כולל את האיבר הנוסף $-g\Delta t^2/2$ לכל צעד. אוילר מזניח איבר זה; השמטות קטנות חוזרות ונשנות מייצרות שגיאת צעד זמן גלובלית.

נשמור את המקום והמהירות בווקטורים בעלי שני רכיבים, $\mathbf{r} = (x, y)$ ו-$\mathbf{v} = (v_x, v_y)$, עם $\mathbf{a} = (0, -g)$. בכל מרווח זמן קצר נחליף את הנגזרת בהפרש סופי:

$$\frac{dx}{dt} \approx \frac{x(t + \Delta t) - x(t)}{\Delta t} \quad \Longrightarrow \quad x(t + \Delta t) \approx x(t) + v_x(t)\Delta t.$$

באותו אופן נקבל $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n\Delta t$ ו-$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$. כל מצב חדש מחושב מהמצב הקודם. ברכיבים, עדכוני המהירות הם $v_{x,n+1} = v_{x,n} + a_{x,n}\Delta t$ ו-$v_{y,n+1} = v_{y,n} + a_{y,n}\Delta t$.

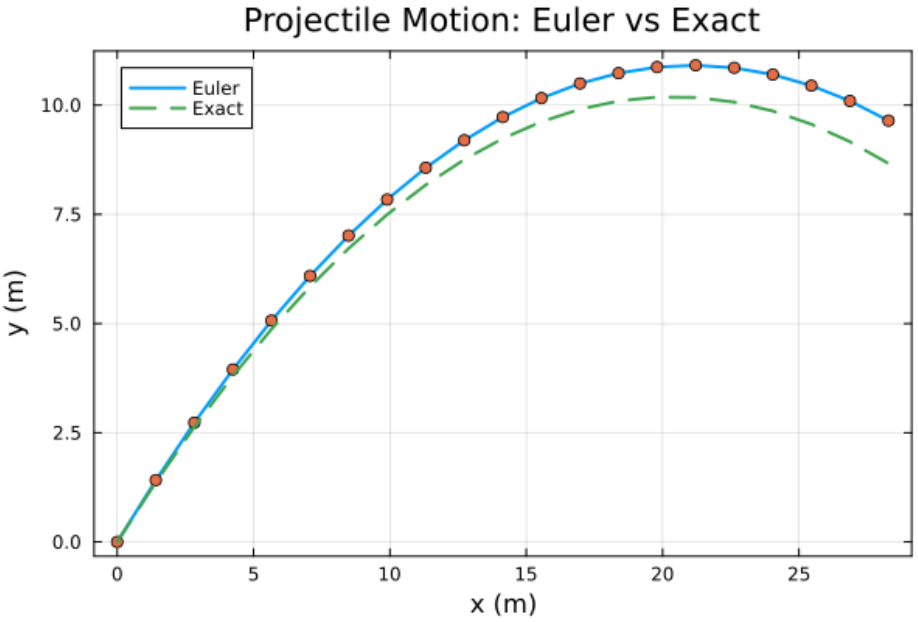
נעקוב אחר הזריקה במשך $T = 2$ שניות, לפני שהיא מגיעה לקרקע. ארבעה וקטורים גדלים שומרים את הרכיבים האופקיים והאנכיים של המקום ושל המהירות. הגרף משתמש בדגימות השמורות; אין צורך לבצע שוב אינטגרציה כדי לצייר אותו. את פרבולת ההשוואה נחשב מהמקום ומהמהירות ההתחלתיים.

הגוש המסופק `let ... end` מחזיק את הסימולציה ואת הגרף שלה באותה סביבת עבודה מקומית. נריץ את הגוש כולו יחד. נחזור למשתנים מקומיים כאשר נציג פונקציות; בינתיים העטיפה מאפשרת לדוגמה המספרית לפעול באופן עקבי בסקריפט ובמחברת. דוגמאות `begin ... end` הקודמות נשארות תרגילים להרצה אינטראקטיבית. `round(Int, T / dt)` נותן את מספר הצעדים השלם, וכאן $T/\Delta t = 20$ בדיוק.

```
using Plots
let
    trajectory_x = Float64[]
    trajectory_y = Float64[]
    velocities_x = Float64[]
    velocities_y = Float64[]
    g = 9.81;         # gravity in m/s^2
    speed = 20.0;     # initial speed in m/s
    theta = pi / 4;  # initial angle in radians
    dt = 0.1;         # time step in seconds
    T = 2.0           # total time in seconds
    r = [0.0, 0.0]   # m
    v = [speed * cos(theta), speed * sin(theta)]   # m/s
    a = [0.0, -g]     # m/s^2
    N = round(Int, T / dt)  # number of time steps
    trajectory_x = [r[1]]
    trajectory_y = [r[2]]
    velocities_x = [v[1]]
```

```
    velocities_y = [v[2]]
    for step in 1:N
        r = r + v * dt
        v = v + a * dt
        push!(trajectory_x, r[1])
        push!(trajectory_y, r[2])
        push!(velocities_x, v[1])
        push!(velocities_y, v[2])
    end
    println("Euler final position: ", round(r[1]; digits=4), ", ", round
(r[2]; digits=4))
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    p = plot(title="Projectile Motion: Euler vs Exact", xlabel="x (m)",
ylabel="y (m)", frame=:box)
    p = plot!(p, trajectory_x, trajectory_y, label="Euler", lw=2)
    p = scatter!(p, trajectory_x, trajectory_y, label=:none, lw=2)
    exact_trajectory_x = Float64[]
    exact_trajectory_y = Float64[]
    ts = 0:dt:T
    for t in ts
        push!(exact_trajectory_x, r0[1] + v0[1] * t + 0.5 * a[1] * t^2)
        push!(exact_trajectory_y, r0[2] + v0[2] * t + 0.5 * a[2] * t^2)
    end
    p = plot!(p, exact_trajectory_x, exact_trajectory_y, label="Exact",
lw=2, ls=:dash)
    display(p)
end
```

```
Euler final position: 28.2843, 9.6453
```



הסמנים מראים את דגימות אוילר השמורות, והקו המקווקו מראה את המסלול המדויק בכבידה קבועה. ב-$t = T$ גובה אוילר הוא 9.6453 מטרים, לעומת הגובה המדויק 8.6643 מטרים. ההפרש הוא $gT\Delta t/2 = 0.981$ מטרים. הקווים המחברים בין הדגימות הם עזר חזותי. בדוגמה הבאה נבדוק כיצד ההפרש משתנה כשמקטינים את צעד הזמן.

3.6 עידון: השוואה לתשובה ידועה

עבור מקום התחלתי $\mathbf{r}_0$ ומהירות התחלתית $\mathbf{v}_0$, התנועה המדויקת בתאוצה קבועה היא

$$\mathbf{r}_{\text{exact}}(t) = \mathbf{r}_0 + \mathbf{v}_0 t + \tfrac{1}{2}\mathbf{a}t^2.$$

נהריץ את אותה אינטגרציה עבור כמה צעדי זמן, תוך שמירה על $T/\Delta t$ שלם:

```
using LinearAlgebra
let
    g = 9.81; speed = 20.0; theta = pi / 4; T = 2.0
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    a = [0.0, -g]
    for dt in (0.2, 0.1, 0.05, 0.02)
        r = copy(r0); v = copy(v0)
        N = round(Int, T / dt)
        for step in 1:N
            r = r + v * dt
            v = v + a * dt
        end
        exact = r0 + v0 * T + 0.5 * a * T^2
        println("dt = $dt, error = ", round(norm(r - exact); digits=4),
" m")
    end
end
```

```
dt = 0.2, error = 1.962 m
dt = 0.1, error = 0.981 m
dt = 0.05, error = 0.4905 m
dt = 0.02, error = 0.1962 m
```

הקטנת Δt בחצי מחצית את שגיאת המקום. בתאוצה קבועה ובאוילר מפורש, המהירות בזמנים הבדידים מדויקת עד לעיגול נקודה צפה, ואילו המקום נושא שגיאה שיטתית:

$$\mathbf{r}_N - \mathbf{r}_{\text{exact}}(T) = -\tfrac{1}{2}\mathbf{a}\,T\Delta t.$$

בכבידה מטה, המסלול המספרי נמצא מעל המסלול המדויק בזמן קבוע, וגודל השגיאה הוא $gT\Delta t/2$, בדיוק הערכים שהודפסו למעלה.

מדוע השגיאה מתרחבת עם Δt? כל עדכון מקום של אוילר משמיט את האיבר $\frac{1}{2}a(\Delta t)^2$, כך ששגיאת המקום המקומית פרופורציונלית ל-$(\Delta t)^2$. ישנם $N = T/\Delta t$ צעדים. בדוגמת התאוצה הקבועה הזאת התרומות המושמטות מצטברות באופן קוהרנטי, ונותנות שגיאה גלובלית פרופורציונלית ל-$N(\Delta t)^2 = T\Delta t$. זו התכנסות מסדר ראשון. הסימולציה אינה הפיסיקה המדויקת, אך השגיאה מתכווצת בצורה צפויה, ואנחנו יכולים לראות אותה מתכווצת.

שאלת סיכום: מה קורה לשגיאה אם נעשה את צעד הזמן קטן פי עשר? בדקו זאת מספרית לפני ההסבר.

אינטגרציה מסדר גבוה יותר: מבט ראשון (רשות)

שגיאת הסדר הראשון של אוילר אינה סוף הסיפור. רעיון טוב יותר: לקחת חצי צעד, להעריך את הנגזרת באותה נקודת אמצע, ולהשתמש בה לצעד המלא:

$$k_1 = f(y_n), \qquad k_2 = f(y_n + \tfrac{1}{2}\Delta t\, k_1), \qquad y_{n+1} = y_n + \Delta t\, k_2.$$

שיטה המשתמשת בנגזרת אחת בטווח הביניים היא שיטה מסדר שני; בתאוצה קבועה היא במקרה מדויקת. לקיחת דגימות נוספות של הנגזרת במקומות חכמים נותנת סדרים גבוהים יותר, עד לשיטת

Runge–Kutta הקלאסית מסדר רביעי (“RK4”). סכמות אלה הן רקע אופציונלי, לא משהו שעלינו לכתוב בקורס זה.

3.7 תרגול: זמן קירור נדגם ראשון

אנו משתמשים במודל הקירור של ניוטון, $\dot{T} = -k(T - T_{\text{room}})$, עבור גוף עם טמפרטורה פנימית אחידה בקירוב בסביבה קבועה. הקצב $k > 0$ הוא ב-s^{-1}; הפרשי טמפרטורה ניתנים למדידה בקלווין או במעלות צלזיוס. אנו מניחים שאיבוד החום פרופורציונלי בקירוב להפרש זה, במקום לטעון שהמודל מכסה אידוי או השפעות קרינה גדולות.

המחברת של התרגול מספקת את העדכון הפיסיקלי. אנו מזהים אתחול, תנאי, עדכון ומונה, ואז משתמשים בלולאת `while` כדי למצוא את הדגימה הראשונה על הסף או מתחתיו. שוויון נחשב הגעה לסף. אנו חוזים מה קורה אם הדגימה הראשונית כבר מתאימה, ומדוע סף מתחת לטמפרטורת החדר אינו ניתן להשגה על ידי קירור מונוטוני מלמעלה. אחרי דיון של 20 דקות אנו מתחילים את תרגיל הבית של פטיש, שהלולאה שלו עוצרת בדגימה הראשונה מתחת לקרקע.

3.8 תרגיל בית ראשי: זריקת פטיש עם התנגדות אוויר

אנו מחקים זריקת פטיש דו-ממדית מהראשית עם התנגדות אוויר ריבועית. המודל ליחידת מסה הוא

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{v}_x = -b v v_x, \quad \dot{v}_y = -g - b v v_y, \qquad v = \sqrt{v_x^2 + v_y^2},$$

כאשר $g = 9.81\ \text{m/s}^2$ ומקדם גרר $b \approx 0.001\ \text{m}^{-1}$. התוכנית קוראת את המהירות ההתחלתית, זווית הזריקה במעלות, הזמן הכולל T, צעד הזמן Δt, ואת b, מקדמת את המצב עם אוילר מפורש ומאחסנת היסטוריות מיושרות של x, y, v_x, v_y. היא עוצרת בדגימה המוצעת הראשונה עם $y < 0$, או אחרי T, ומדפיסה את המרחק האופקי שהושג.

תחילה נשרטט את המסלול הצפוי ונציין יחידות וסימנים. אחר כך נבנה את הווקטורים ואת הלולאה, תוך הקפדה על עדכון מקום מהמהירות הישנה. נבדוק את העדכון הראשון ביד ונאמת שכל וקטור גדל באיבר אחד לכל צעד, שהמהירות האופקית דועכת רק דרך גרר, ושהתאוצה האנכית כוללת גם כבידה וגם את איבר הגרר. נריץ את התוכנית עם הדוגמה הנתונה, נשווה את המרחק המודפס לאומדן שחושב ביד, ונעדן את Δt כדי לראות כיצד המרחק הנדגם זז. נשמור את גרף המסלול כמוצר. הדו”ח חייב לציין איזה כלל עצר את הלולאה: חציית הקרקע או סוף חלון הסימולציה. ההרחבה האופציונלית בגיליון, הבודקת האם 45° עדיין ממקסמת את הטווח, אינה נדרשת. הגדרת פונקציות נותרת חומר של שבוע 5.

3.9 בונוס אופציונלי: שלוש סכמות אינטגרציה

בונוס זה הוא אופציונלי, אינו תנאי מוקדם לעבודה מאוחרת יותר, ואינו נבחן. הוא שווה **שליש נקודה בציון הסופי**.

אנו משווים אוילר קדמי, אוילר-קרומר, וסכמת velocity-Verlet (נקודת אמצע) על נפילה חופשית באמצעות מעקב אחר האנרגיה המכנית ליחידת מסה. אנו קוראים את הזמן הסופי, צעד הזמן, הגובה ההתחלתי והמהירות ההתחלתית, מריצים את שלוש הסכמות על אותה רשת ומדפיסים את שלוש האנרגיות הסגוליות הסופיות. לאחר מכן נשרטט את האנרגיה הסגולית כפונקציה של הזמן עבור שלוש הסכמות יחד עם הערך השמור המדויק. שתי שאלות תיאורטיות קצרות מבקשות מאתנו לגזור את שינוי האנרגיה לכל צעד עבור כל סכמה ולומר איזו סכמה עדיפה ומדוע.

3.10 המשך חקר: מהליכה אחת לדיפוזיה

מסלול אחד יכול לנדוד חזרה לכיוון המוצא. דיפוזיה היא הצהרת אנסמבל: עבור צעדים יחידתיים בלתי תלויים ונטולי הטיה, ההעתק הריבועי הממוצע מקיים $\langle r_n^2 \rangle = n$ ביחידות סריג, כי איברי ההצלבה מתאפסים בממוצע. אם צעד אחד נמשך τ ואורכו ℓ, מקדם הדיפוזיה הדו-ממדי המקביל הוא $D = \ell^2/(4\tau)$.

באמצעות מתכון אנסמבל מסופק, אנו משווים אוסף קטן וגדול יותר של הליכות, משרטטים העתק ריבועי ממוצע כנגד מספר הצעד, ובוחנים תנודות סביב הקו הישר הצפוי. אנו מסבירים מדוע הריבוע של

ההעתק הממוצע הוא גודל שונה ויכול להישאר קרוב לאפס. אנו מתעדים את הגדרת הגנרטור ואת גודל האנסמבל עם הגרף. חקר זה הוא אופציונלי ואינו נבחן; מכונות האנסמבל המסופקת אינה תנאי מוקדם לעבודה ראשית.



4. מטריצות, לולאות וסיבובים

גדלים רבים בפיסיקה אינם מספר יחיד אלא טבלה מאורגנת של מספרים. מטריצה שומרת טבלה כזאת, וסיבוב הוא אחת ההתמרות החשובות שאפשר להפעיל עליה. נלמד לקרוא מטריצה ולבחון את צורתה, נשתמש בלולאות כדי למלא מטריצות ולשלב ביניהן, ולבסוף נסובב בעזרת מטריצות צורות וקטורים. לאורך הפרק נכתוב בעצמנו את החישוב, איבר אחר איבר, כדי להבין מה הפעולות המובנות עושות. נגדיר פונקציות משלנו בשבוע 5; כאן נכתוב הכול בלולאות ובתחומי העבודה `let`.

4.1 כתיבת מטריצה וצורתה

מטריצה היא מערך מלבני של מספרים המסודרים בשורות ובעמודות. בכתיבת מטריצה ב־Julia, רווחים מפרידים בין איברי אותה שורה, ונקודה־פסיק מפרידה בין השורות:

```
A = [1 2 3; 4 5 6]      # a 2-by-3 matrix: 2 rows, 3 columns
println(A)
println(size(A))         # (2, 3)
println(size(A, 1))      # 2   (number of rows)
println(size(A, 2))      # 3   (number of columns)
println(length(A))       # 6   (total number of entries)
```

```
[1 2 3; 4 5 6]
(2, 3)
2
3
6
```

למטריצה `A` שתי שורות ושלוש עמודות, ולכן צורתה היא "2 על 3". הפונקציה `size(A)` מחזירה את זוג הממדים; `size(A, 1)` מחזירה את מספר השורות, ו־`size(A, 2)` את מספר העמודות. מכפלת הממדים נותנת את מספר האיברים הכולל, שאותו מחזירה `length`. בדרך כלל נתחיל בבדיקת הצורה: אפשר לשלב שתי מטריצות רק כאשר ממדיהן מתאימים לפעולה.

4.1.1 אינדוקס מטריצה

נאתר איבר יחיד באמצעות אינדקס השורה תחילה, ואחריו אינדקס העמודה:

```
println(A[2, 3])         # 6
println(A[1, :])         # [1 2 3]   the first row (a 1-by-3 matrix)
println(A[:, 2])         # [2, 5]    the second column (a vector)
```

```
6
[1 2 3]
[2, 5]
```

הביטוי `A[2, 3]` בוחר את האיבר בשורה 2 ובעמודה 3. נקודתיים בוחרות את כל האיברים לאורך אותו ציר: `A[1, :]` היא השורה הראשונה כולה, ו־`A[:, 2]` היא העמודה השנייה כולה. שימו לב להבדל בפלט: שורה שומרת על הפריסה האופקית שלה, ואילו עמודה מוצגת כווקטור רגיל. גם מטריצה בעלת עמודה אחת נשארת מטריצה, ו־Julia מציגה אותה בהתאם.

תחילה שורות, אחר כך עמודות

בכל ביטוי אינדוקס מציינים את השורה לפני העמודה: `A[i, j]` פירושו שורה i ועמודה j. נקודתיים `:` פירושן "כל האינדקסים לאורך הציר הזה". בחירה של שורה או עמודה מחזירה תמיד תוצאה בכיוון המתאים, שאפשר להשתמש בה ישירות במכפלה או בגרף.

4.2 לולאות while: חזרה עד שתנאי נעשה שקרי

לעיתים איננו יודעים מראש כמה פעמים נצטרך לחזור על פעולה. לולאת `while` ממשיכה כל עוד התנאי שלה אמת, ובודקת אותו לפני כל איטרציה. בניגוד ללולאת `for`, אין לה מונה מובנה; לכן משהו בגוף הלולאה צריך לשנות את התנאי, אחרת היא לא תסתיים.

```
let
    n = 0
    while n < 5
        n += 1
        println(n)
    end
end                     # prints 1, 2, 3, 4, 5
```

```
1
2
3
4
5
```

כאן `n` הוא המונה, `n += 1` הוא העדכון, ו־`n < 5` הוא התנאי. הלולאה בודקת את התנאי, מבצעת את הגוף, בודקת שוב ועוצרת ברגע שהתנאי אינו מתקיים.

4.2.1 קלט עם ערך סיום: ממוצע הכיתה

שימוש נפוץ בלולאת `while` הוא קריאת קלט עד שמופיע ערך סיום מיוחד. בדוגמה נקרא ציונים עד שהמשתמש יקליד -1. ערך הסיום מציין שהקלט הסתיים; הוא אינו ציון.

```
let
    sum_grades = 0      # total of the grades
    n_grades = 0        # how many grades were entered
    print("Enter grade (-1 to stop): ")
    grade = parse(Int, readline())
    while grade != -1
        sum_grades += grade
        n_grades += 1
        print("Enter grade (-1 to stop): ")
        grade = parse(Int, readline())
    end
    if n_grades != 0
        avg = sum_grades / n_grades
```

```
        println("Class average is ", round(avg; digits = 2),
                "; number of grades is ", n_grades)
    else
        println("No grades were entered")
    end
end
```

הקריאה הראשונה מתבצעת לפני הלולאה, ובסוף כל איטרציה קוראים שוב. אם המשתמש מקליד מיד -1, גוף הלולאה כלל אינו מתבצע, ואנו מדווחים שלא הוזנו ציונים במקום לחלק באפס.

ערך סיום הוא חלק מכללי הקלט

ערך סיום מסמן את סוף רצף הקלט, ולכן צריך להיות אפשר להבחין בינו לבין נתונים תקפים. כאן -1 מתאים, כי ציונים נמצאים בין 0 ל־100. התנאי של `while` בודק אם הגיע ערך הסיום; הוא אינו מונה את האיטרציות.

4.2.2 בדיקת תקינות הקלט

תבנית נפוצה נוספת ממשיכה לבקש קלט עד שהתשובה תקינה. הלולאה נמשכת כל עוד הקלט אינו תקין, ולכן התנאי מתאר את ההפך מן המצב הרצוי:

```
let
    print("Please enter a positive number 20..100: ")
    flush(stdout) # Make sure the prompt appears before waiting for
input.
    number = parse(Int, readline())
    while number <= 20 || number >= 100
        print("A positive number 20..100, try again: ")
        number = parse(Int, readline())
    end
    println("Your positive number is ", number)
end
```

התנאי `number <= 20 || number >= 100` אומר "עדיין לא תקין". רק ערך שנמצא ממש בין 20 ל־100 מסיים את הלולאה. שימו לב לגבולות: האופרטורים => ו־=< דוחים בדיוק את 20 ואת 100. כדי לקבל גם אותם נצטרך להשתמש ב־> וב־<. הקריאה `flush(stdout)` דואגת שההנחיה תופיע לפני שהתוכנית ממתינה לקלט, דבר שחשוב במסופים שאוגרים פלט.

4.2.3 משחק ניחושים

משחק הניחושים משלב את שתי התבניות. המחשב בוחר מספר, ואנו קוראים ניחושים ומקבלים רמז בכל פעם, עד שאחד מהם מתאים:

```
let
    number = rand(1:16)
    print("please guess my number between 1 and 16 ")
    flush(stdout) #
    guess = parse(Int, readline())
    while guess != number
        if number > guess
            print("My number is bigger, please try again ")
        else
            print("My number is smaller, please try again ")
        end
        flush(stdout)
        guess = parse(Int, readline())
    end
    println("You guessed it! The number is ", number)
```

```
end
```

ייתכן שהלולאה לא תתבצע כלל אם הניחוש הראשון נכון, או שתתבצע פעמים רבות. כל ניחוש שגוי מספק רמז שמצמצם את טווח החיפוש; לכן בדרך כלל די בכמה ניסיונות כדי למצוא מספר מתוך 16 אפשרויות.

4.2.4 מדידת זמן של לולאת while: הטור ההרמוני

הטור ההרמוני $1 + \frac{1}{2} + \frac{1}{3} + \dots$ גדל באיטיות רבה, בקירוב כמו $\ln n$. לכן דרושים איברים רבים כדי להגיע אפילו לסכום מתון, והטור מאפשר לנו להתרשם מקצב הריצה של Julia. נוסיף איברים עד שהסכום יעבור סף שנבחר, ונמדוד את זמן הלולאה:

```
let
    print("Enter positive limit: ")
    flush(stdout)
    limit = parse(Float64, readline())

    max_terms = 100_000_000   # safety cap on the number of terms
    t1 = time_ns()
    series_sum = 0.0
    n = 0
    while series_sum <= limit
        n += 1
        series_sum += 1.0 / n
        if n >= max_terms
            println("Limit ", limit, " too large: stopped after ", n,
                    " terms without exceeding it.")
            break
        end
    end
    t2 = time_ns()

    println("Number of terms needed: ", n)
    println("Elapsed seconds: ", round((t2 - t1) / 1.0e9; digits = 3))
    # For limit 10: 12367 terms (instant).
    # For limit 20: about 272,400,600 terms — more than the cap, so the
loop
    # stops at 100,000,000 terms and prints that the limit was too large.

end
```

בסף 10 נדרשים 12367 איברים, והחישוב מסתיים מיד. בסף 20 יידרשו בערך 272 מיליון איברים — יותר מתקרת הבטיחות שלנו, מאה מיליון — ולכן הלולאה נעצרת מוקדם ומדווחת שהסף גדול מדי. כדאי להתרגל לשומר `if n >= max_terms` ולפקודת `break`: אם לולאה עלולה להימשך זמן רב, נקבע לה גבול ונחליט מה לעשות כשנגיע אליו. הזמנים נמדדים באמצעות `time_ns()`, שקוראת שעון ברזולוציה של ננו־שניות; חלוקת הפרש הזמנים ב־10^9 נותנת שניות.

מתכננים מראש כיצד הלולאה תסתיים

בלולאת `while` נגדיר ערך התחלתי, תנאי שיכול להפוך לשקרי, ועדכון שמשנה אותו. אם אחד מהם חסר, הלולאה עלולה להימשך בלי סוף. חסם על מספר האיטרציות מגן מפני תקלה והופך חיפוש בלתי מוגבל לחישוב סופי.

הדוגמאות האינטראקטיביות האלה פועלות באופן אמין יותר כקבצים עצמאיים מאשר בהרצה תא אחר תא בעורך, ולכן ארבע דוגמאות הקלט נאספו גם בתיקייה week04_readline_examples שבתוך examples. מתוך סביבת REPL, אחרי מעבר לתיקיית examples, אפשר להריץ כך: `include("week04_readline_examples/class_average_sentinel.jl")`; נעשה גם עבור שלושת הקבצים האחרים.

4.3 מכפלת מטריצה בווקטור ובמטריצה

מטריצה יכולה לפעול על וקטור או על מטריצה אחרת, ו־Julia מסמנת את המכפלות האלה באופרטור הרגיל *:

```
M = [1 2; 3 4]
v = [5, 6]
println(M * v)          # [17, 39]   matrix * vector: linear
combination of columns
println(M .* M)         # [1 4; 9 16]  elementwise multiplication (
dotted)
println(M * M)          # [7 10; 15 22]  true matrix product
```

```
[17, 39]
[1 4; 9 16]
[7 10; 15 22]
```

השורה הראשונה מציגה מכפלת מטריצה בווקטור: עמודות M משוקללות לפי איברי v, והתוצאה היא וקטור חדש. בשורה השנייה מופיעה כפל איבר־איבר, המסומן בנקודה: כל איבר של M מוכפל באיבר שנמצא באותו מקום. השורה השלישית היא מכפלת מטריצות רגילה; בעוד רגע נבנה אותה בעצמנו בעזרת לולאות.

כפל מטריצות משלב שורה ועמודה

האיבר בשורה i ובעמודה j של המכפלה A * B הוא המכפלה הסקלרית של שורה i ב־A עם עמודה j ב־B:

$$(AB)_{ij} = \sum_{k} A_{ik} B_{kj}.$$

כדי שהסכום יהיה מוגדר, הממדים הפנימיים צריכים להיות שווים: מספר העמודות של A צריך להיות כמספר השורות של B. לעומת זאת, הכפל איבר־איבר A .* B מצמיד איברים באותם מקומות ודורש צורות זהות. הנקודה מסמנת פעולה אחרת, עם כללי חישוב משלה.

4.4 לולאות מקוננות: לוח כפל

לולאה חוזרת על גוש הוראות. בלולאה מקוננת לולאה אחת נמצאת בתוך אחרת, ולכן הפנימית משלימה את ריצתה עבור כל ערך של הלולאה החיצונית. כך נוח למלא מטריצה שורה אחר שורה. דוגמה פשוטה היא לוח כפל בגודל 10×10, שבו כל איבר table[i, j] שומר את המכפלה i * j:

```
let
    table = zeros(Int, 10, 10)
    for i in 1:10
        for j in 1:10
            table[i, j] = i * j
        end
    end
    println(table[3, 4])       # 12
    println(table[10, 10])     # 100
    println(table[3, 1:5])     # [3, 6, 9, 12, 15]   (row 3)
    println(table)
end
```

```
12
100
[3, 6, 9, 12, 15]
```

נעקוב אחר סדר הלולאות. הלולאה החיצונית קובעת תחילה $i = 1$, והפנימית עוברת על $j = 1, \ldots, 10$ וממלאת את השורה הראשונה. אחר כך i נעשה 2, והלולאה הפנימית ממלאת את השורה השנייה. המשתנה הפנימי משתנה מהר יותר, כפי שאנו קוראים טבלה: משמאל לימין, שורה אחר שורה. הפקודה האחרונה `println(table)` מדפיסה את כל המטריצה בסימון התמציתי של Julia, בשורה ארוכה אחת שבה נקודה־פסיק מפרידה בין השורות. הפלט נכון, אך קשה מאוד לקרוא אותו.

הדפסה קריאה של מטריצה (רשות)

הדפסת המטריצה הרגילה מספיקה כאן, אך מטריצות רחבות קשות לקריאה. אם נרצה עמודות מיושרות, החבילה PrettyTables.jl מספקת את `pretty_table(table)`, המציירת טבלה עם מסגרת. החבילה אינה כלולה בסביבת הקורס ואינה דרושה לתרגילים; התקנת חבילות נוספות אינה חלק מהקורס, ולכן נמשיך להשתמש בהדפסה הרגילה של המטריצה.

4.5 לולאות מקוננות: כפל מטריצות

כעת נחשב את המכפלה בעצמנו. כל איבר בתוצאה הוא מכפלה סקלרית של שורה בעמודה, ולכן שתי לולאות עוברות על כל איברי התוצאה ולולאה פנימית צוברת את הסכום לאורך הממד המשותף. כאשר מטריצה ארוכה מכדי להיכתב בנוחות בשורה אחת, נפרוס אותה על פני כמה שורות וניישר אותן כדי לשמור על צורתה ברורה:

```
let
    A = [1.0 2.0 3.0;
         4.0 5.0 6.0]   # 2-by-3
    B = [1.0 0.0 2.0 1.0;
         0.0 1.0 1.0 0.0;
         2.0 1.0 0.0 1.0]   # 3-by-4
    m, k = size(A)
    k2, n = size(B)
    @assert k == k2 "inner dimensions must match"

    C = zeros(m, n)
    for i in 1:m          # output row
        for j in 1:n      # output column
            for r in 1:k  # the shared inner dimension
                C[i, j] += A[i, r] * B[r, j]
            end
        end
    end

    println(C)             # 7.0  5.0  4.0  4.0  /  16.0  11.0  13.0
10.0
    println(A * B)         # same numbers, built by Julia's own matrix
product
    println(isapprox(C, A * B))   # true
end
```

```
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
true
```

זוהי לולאה משולשת. שתי הלולאות החיצוניות בוחרות את איבר התוצאה (i, j), והפנימית מחברת את $A[i,1]B[1,j] + A[i,2]B[2,j] + \ldots$ עד שמוצו הממדים המשותפים. פריסת המטריצות על כמה שורות אינה משנה את הערכים; היא רק משאירה את מבנה השורות קריא, וההערה שבסוף מזכירה לנו את הצורה שכתבנו. שורת `@assert` משמשת כשומר: אם הממדים הפנימיים אינם מתאימים, התוכנית נעצרת עם

הודעה ברורה במקום לחשב תוצאה חסרת משמעות. התאמת התוצאה למכפלה `A * B` של Julia מאשרת שהבנו את הכלל.

4.6 בדיקה אם מטריצה סימטרית

מטריצה סימטרית אם $A_{ij} = A_{ji}$ לכל זוג אינדקסים: האיבר בשורה i ובעמודה j שווה לאיבר בשורה j ובעמודה i. נוכל לבדוק זאת בלולאה מקוננת. מספיק לבדוק את המשולש העליון, מפני שכל איבר באלכסון שווה לעצמו:

```
let
    M = [1 2 3;
         2 5 6;
         3 6 9]    # symmetric by construction
    symmetric = true
    for i in 1:size(M, 1)
        for j in i:size(M, 2)    # j starts at i, so we visit the upper
triangle
            if M[i, j] != M[j, i]
                symmetric = false
            end
        end
    end
    println("Matrix M is symmetric: $symmetric")           # true

    N = [1 2 3;
         4 5 6;
         7 8 9]   # not symmetric
    symmetric = true
    for i in 1:size(N, 1), j in i:size(N, 2)
        if N[i, j] != N[j, i]
            symmetric = false
        end
    end
    println("Matrix N is symmetric: $symmetric")           # false
end
```

```
Matrix M is symmetric: true
Matrix N is symmetric: false
```

שתי המטריצות כתובות שורה בכל שורה, ולכן הסימטריה של `M` נראית מיד: המספרים לאורך שורה זהים למספרים לאורך העמודה המתאימה. הלולאה בודקת את התכונה ישירות. התחלת הלולאה הפנימית ב־`j in i:…` מדלגת על המשולש התחתון ועל האלכסון, וכך מקצרת בערך בחצי את העבודה. הלולאה השנייה משתמשת בצורה התמציתית `for i in …, j in …`, השקולה לשתי לולאות מקוננות.

יש גם בדיקה קצרה: השחלוף `M'` מחליף בין שורות לעמודות, ולכן מטריצה סימטרית בדיוק כאשר `M == M'`. במטריצה כמעט סימטרית שאיבריה מספרי נקודה צפה, בדיקה זו נכשלת בגלל עיגול שמשנה זוג איברים בכ־10^{-14}. במקרה כזה נשתמש ב־`isapprox` עם סבילות, כפי שמודגם בקטע האחרון.

```
# A shortcut: M' is the transpose. Comparing M with its transpose is the
 same check.
M = [1 2 3; 2 5 6; 3 6 9]
N = [1 2 3; 4 5 6; 7 8 9]
println(M == M')         # true
println(N == N')         # false
```

```
true
false
```

נבצע את אותה השוואה במספרי נקודה צפה, הפעם עם סבילות:

```
let
    P = [1.0 2.0;
         2.0 1.0]
    Q = P + [0.0 1e-14;
             0.0 0.0]   # nearly symmetric, but not exactly
    println(P == P')               # true
    println(Q == Q')               # false  (machine noise breaks exact
 equality)
    println(isapprox(Q, Q'; atol = 1e-12))    # true
    println(maximum(abs.(Q - Q')))            # 1.0e-14
end
```

```
true
false
true
1.0e-14
```

`Q` שונה מהשחלוף שלה ב־10^{-14} בלבד — הפרש זעיר מבחינה פיסיקלית, אך די בו כדי שהשוואה באמצעות `==` תיכשל. `isapprox` עם `atol = 1e-12` מקבלת את המטריצה. הביטוי `maximum(abs.(Q - Q'))` מדווח ישירות על גודל ההפרש; הנקודה ב־`abs.` מפעילה ערך מוחלט על כל איבר בנפרד.

4.7 מטריצות סיבוב בשני ממדים

מטריצה המסובבת וקטור נגד כיוון השעון בזווית θ היא

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}.$$

כאשר הווקטורים הם עמודות, הנקודה המסובבת היא $R\mathbf{p}$. סיבוב הווקטור $(1,0)$ ב־90° אמור להביא אותו ל־$(0,1)$:

```
using LinearAlgebra
let
    θ = π / 2
    R = [cosθ() -sinθ();
         sinθ() cosθ()]
    println(R)                         # 0.0  -1.0  /  1.0  0.0
    p = [1.0, 0.0]
    println(R * p)                     # [0.0, 1.0]   (a quarter turn)
end
```

```
[0.0 -1.0; 1.0 0.0]
[0.0, 1.0]
```

המטריצה כתובה כך שהשורה השנייה מופיעה בשורה נפרדת, ולכן שתי השורות נשארות ברורות. Julia מחזירה את האיברים כ־`6.12e-17` ולא כאפס מדויק, מפני שאי אפשר לייצג את $\cos(\pi/2)$ בדיוק במספרי נקודה צפה; בהדפסה הערך מעוגל ל־`0.0`. השארית הזעירה הזאת מסבירה מדוע בקטע הבא נשווה גדלים בעזרת סבילות ולא באמצעות `==`.

4.7.1 בדיקה שמטריצה אכן מתארת סיבוב

שתי תכונות מבדילות סיבוב ממטריצה כללית. סיבוב שומר על מרחקים, ולכן המרחק בין שתי נקודות אינו משתנה. השחלוף של מטריצת הסיבוב הוא המטריצה ההפוכה שלה, $R'R = I$, והיא שומרת גם על הכיווניות, כך שהדטרמיננטה שלה היא 1. נבדוק ישירות את שתי התכונות הראשונות:

```
let
    p = [1.0, 2.0]
    q = [-0.5, 3.0]
    θ = π / 6
    R = [cosθ() -sinθ();
         sinθ() cosθ()]

    println(isapprox(R' * R, I; atol = 1e-14))   # true  (orthogonal)

    d_before = norm(p - q)         # distance before rotation
    d_after = norm(R * p - R * q)  # distance after rotation
    println(round(d_before; digits = 6))         # 1.802776
    println(round(d_after; digits = 6))          # 1.802776   (same
distance)
end
```

```
true
1.802776
1.802776
```

המכפלה R' * R מתקבלת כמטריצת היחידה בדיוק של 10^{-14}, והמרחק בין הנקודות לאחר הסיבוב שווה למרחק המקורי. כשנבנה התמרה שאמורה לתאר סיבוב, שתי הבדיקות האלה יאפשרו לנו לברר זאת במהירות.

4.8 סיבוב מלבן במישור

כדי לסובב צורה, נסובב כל קודקוד ונחבר מחדש את הנקודות שהתקבלו. נשמור את ארבע פינות המלבן בשורות של מטריצה. בדוגמה נסובב את המלבן בזוויות $0°, 30°, 60°, 90°$, ואחרי כל סיבוב נדפיס את מיקומו של אחד הקודקודים:

```
let
    θs = (0.0, 30.0, 60.0, 90.0)
    rect = [0.0 0.0;
            2.0 0.0;
            2.0 1.0;
            0.0 1.0]
    for θ in θs
        R = [cosd(θ) -sind(θ);
             sind(θ) cosd(θ)]
        rotated = zeros(size(rect))  # same shape as rect
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]   # the corner
as a vector
            moved = R * corner                            # the rotated
corner
            rotated[vertex, 1] = moved[1]
            rotated[vertex, 2] = moved[2]
        end
        println(θ," = ", θ, "°  corner (2,0) -> [", round(rotated[2, 1];
digits = 3),
                ", ", round(rotated[2, 2]; digits = 3), "]")
    end
    # θ = 0.0°   corner (2,0) -> [2.0, 0.0]
    # θ = 30.0°  corner (2,0) -> [1.732, 1.0]
    # θ = 60.0°  corner (2,0) -> [1.0, 1.732]
```

```
    # θ = 90.0°  corner (2,0) -> [0.0, 2.0]
end
```

כל שורה ב־rect מייצגת קודקוד אחד; כתיבת שורה נפרדת לכל קודקוד עוזרת להבחין בין ארבעתם. הלולאה הפנימית קוראת את שתי הקואורדינטות של קודקוד אחד בעזרת אינדקסים בודדים, rect[vertex, 1] ו־rect[vertex, 2], אורזת אותן בווקטור בן שני איברים, ומכפילה את המטריצה בווקטור הזה. זוהי בדיוק מכפלת מטריצה בווקטור מהסעיף הקודם, המופעלת פעם אחת לכל קודקוד, ולכן אין צורך לדעת דבר על פרוסות. שתי הקואורדינטות של התוצאה נכתבות לשורה המתאימה ב־rotated, שאותו zeros(size(rect)) כבר הכין באותה צורה. הקודקוד $(2, 0)$ נע לאורך מעגל שרדיוסו 2, עובר דרך $(1.732, 1.0)$ בזווית $30°$, וב־$90°$ מגיע לציר y — בדיוק כפי שמצופה מסיבוב של רבע מעגל. הפונקציות cosd ו־sind מקבלות זוויות במעלות, ולכן אין צורך בהמרה מפורשת.

ציור כל העותקים יחד ממחיש את הסיבוב טוב יותר מקריאת הקואורדינטות:

```
using Plots
let
    rect = [0.0 0.0; 2.0 0.0; 2.0 1.0; 0.0 1.0]

    p = plot(title = "Rectangle rotated in the xy plane",
             xlabel = "x", ylabel = "y",
             legend = false, aspect_ratio = :equal)

    for θ in 0:30:330
        R = [cosd(θ) -sind(θ); sind(θ) cosd(θ)]
        xs = Float64[]; ys = Float64[]
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]
            moved = R * corner
            push!(xs, moved[1])
            push!(ys, moved[2])
        end
        # Close the polygon back to its first vertex.
        push!(xs, xs[1]); push!(ys, ys[1])
        plot!(p, xs, ys; lw = 2)
    end

    display(p)
end
```

שני פרטים בקטע הזה ראויים לתשומת לב. הלולאה החיצונית עוברת בזוויות מ־$0°$ עד $330°$ בצעדים של 30 מעלות, ובכל איטרציה מציירת מלבן מסובב. בתוך הלולאה כל קודקוד הופך לווקטור בן שני איברים, מסובב באותה מכפלת מטריצה בווקטור כמו קודם, ושתי הקואורדינטות שלו נאספות בווקטורים xs ו־ys בעזרת push!, ולא באמצעות הגדרת וקטור ברשימה, כדי להמשיך לתרגל לולאות מפורשות. הוספת הנקודה המחושבת הראשונה פעם נוספת סוגרת את המצולע; בלעדיה הצלע האחרונה חסרה והצורה נראית כמו מסלול פתוח.



5. פונקציות בפיסיקה

במשך חודש בנינו חישובים מלולאות, וקטורים ומטריצות. בכל בעיה חדשה העתקנו קטע קוד ושינינו בו ערך או שניים. השבוע נארוז חישוב פעם אחת, ניתן לו שם, ונקרא לו בכל פעם שנזדקק לו. זה תפקידה של פונקציה, ובעזרתה התוכנית שלנו הופכת לכלי שאפשר להשתמש בו שוב.

הדוגמה הפיסיקלית שלנו היא מטוטלת שמשתחררת מזווית גדולה. אי אפשר לבטא את המחזור שלה באמצעות פונקציות אלמנטריות, ולכן נפתור מספרית את משוואת התנועה. אותה פונקציה תאפשר לנו לבדוק כיצד המחזור תלוי בזווית השחרור — קשר שכדאי לשרטט ולחקור.

5.1 מחישוב לפונקציה

נתחיל בארגומנט אחד. פונקציה מקבלת את הערכים שאנו מעבירים לה, הנקראים ארגומנטים, פועלת עליהם באופן מקומי ומחזירה לנו תוצאה באמצעות `return`:

```
function double_number(x)
    return 2 * x
end

println(double_number(3))      # 6
```

```
6
```

השם x קיים רק בתוך הפונקציה. אחרי שהפונקציה מחזירה ערך, נוכל לקרוא לה שוב עם מספר אחר, בלי שיישאר דבר מן הקריאה הקודמת. כעת החישוב מרוכז במקום אחד, ולכן לולאה יכולה להשתמש בו ישירות:

```
for n in 1:5
    println(double_number(n))
end
```

```
2
4
6
8
10
```

כדאי להתרגל לחלוקת העבודה הזאת: הלולאה קובעת באילו ערכים נשתמש, והפונקציה קובעת מה נעשה עם ערך אחד.

פונקציה בעלת שני ארגומנטים פועלת באותו אופן. גם כאן השמות בגוף הפונקציה מקומיים:

```
function rectangle_area(width, height)
    area = width * height
    return area
end

println(rectangle_area(3, 4))   # 12
width = 100
area = -1
println(rectangle_area(3, 4))   # 12
println(width)                  # 100
println(area)                   # -1
```

```
12
12
100
-1
```

לפונקציה יש משתנים משלה בשם `width` ו־`area`; השמות שמחוץ לה שומרים על ערכיהם. כאן שני האורכים נמדדים באותה יחידה שרירותית, והשטח המוחזר נמדד בריבוע היחידה הזאת.

אפשר לכתוב חישוב קצר בשורה אחת

כאשר גוף הפונקציה הוא ביטוי יחיד, נוכל לקצר את ההגדרה:

```
square(x) = x * x
println(square(5))       # 25
```

שתי הצורות מגדירות אותה פונקציה. נשתמש בצורה הארוכה כאשר הגוף דורש כמה צעדים או הערה.

5.2 תחום הראות: היכן שם מוגדר

משמעותו של שם נקבעת בתוך תחום ראות. ברמה העליונה של קובץ תוכנית או של סביבת REPL, השמה יוצרת שם גלובלי. בתוך פונקציה היא יוצרת שם מקומי. ההבדל מתגלה ברגע שנשתמש באותו שם בשני המקומות:

```
x = 10
println("Global x = ", x)              # Global x = 10

function scope_function_x()
    x = 20
    return x
end
println("Function's local x = ", scope_function_x())   # 20
println("After function, global x = ", x)              # 10
```

ההשמה בתוך הפונקציה השאירה את `x` הגלובלי ללא שינוי. בדרך כלל זה בדיוק מה שנרצה: ההשמה המקומית אינה משנה מאחורי הקלעים את שאר התוכנית.

גוש `begin` פועל אחרת. הוא מקבץ הוראות בלי ליצור תחום ראות, ולכן ברמה העליונה הוא משנה את אותו שם גלובלי:

```
begin
    x = 30
    println("Inside top-level begin, x = ", x)   # 30
end
println("After begin, global x = ", x)           # 30
```

לעומת זאת, גוש `let` פותח תחום ראות מקומי:

```
let
    x = 40
    println("Inside let, local x = ", x)   # 40
end
println("After let, global x = ", x)       # 30
```

אם נכתוב `begin` בתוך פונקציה, הוא ישתמש בתחום הראות שלה:

```
function scope_function_begin_x()
    x = 50
    begin
        x = 60
    end
    return x
end
println(scope_function_begin_x())   # 60
println(x)                          # 30
```

גוש `begin` משתמש בתחום הראות שמקיף אותו, ולכן עצם כתיבתו אינה הופכת את הגוש לגלובלי. תחום ראות מקומי נוגע לשמות ואינו מעתיק עצמים: שם מקומי עדיין יכול להפנות לאותו מערך שאליו מפנה שם גלובלי, כפי שנראה בסעיפים הבאים.

5.3 ארגומנטים מיקומיים, ארגומנטי מפתח וערכי ברירת מחדל

ארגומנטים מיקומיים מותאמים לפי סדר הופעתם, ולכן הסדר חשוב:

```
function difference(a, b)
    return a - b
end
println(difference(10, 3))    # 7
println(difference(3, 10))    # -7
```

אפשר להגדיר לארגומנט מיקומי ערך ברירת מחדל, שישמש כאשר נשמיט אותו בקריאה:

```
function shifted(x, amount = 1)
    return x + amount
end
println(shifted(5))       # 6
println(shifted(5, 10))   # 15
```

כשיש כמה ערכי ברירת מחדל, הם משלימים את הארגומנטים החסרים מימין; בקריאה אי אפשר לדלג על ארגומנט מיקומי באמצע:

```
function confusing(x, y = 10, z = 100)
    return x + y + z
end
confusing(3, 1)   # 3 + 1 + 100 = 104
confusing(3)      # 3 + 10 + 100 = 113
```

ארגומנטי מפתח מצוינים בשמם בעת הקריאה. נקודה־פסיק בהגדרה מסמנת את תחילתם:

```
function affine(x; scale = 1, offset = 0)
    return scale * x + offset
end
println(affine(3))                          # 3
println(affine(3; scale = 2, offset = 1))   # 7
println(affine(3; offset = 1, scale = 2))   # 7
```

ב־Julia סדר ארגומנטי המפתח אינו משנה, ובקריאה אפשר להחליף את הנקודה־פסיק בפסיק: `affine(3, offset = 1)` מחזירה 4. הנקודה־פסיק נחוצה בהגדרה כדי לסמן היכן מתחילים ארגומנטי המפתח. נשתמש בהם כאשר מספר ללא שם בקריאה עלול להיות לא ברור.

5.4 החזרת ערך

פונקציה ללא `return` מחזירה את ערך הביטוי האחרון שלה:

```
function cube(x)
    x^3
end
println(cube(3))      # 27
```

הפקודה `return` גם יוצאת מן הפונקציה מיד, אפילו מתוך לולאה. כך נוכל לעצור ברגע שהתשובה ידועה, ולדווח על היעדר תשובה באמצעות `nothing`:

```
function first_positive(values)
    for value in values
        if value > 0
            return value
        end
    end
    return nothing
end
println(first_positive([-2, 0, 5, 9]))   # 5
println(first_positive([-2, 0]))         # nothing
```

`nothing` הוא ערך בפני עצמו, השונה מן המספר אפס וממחרוזת ריקה. אפשר לבדוק אותו ישירות. ב־Julia יש גם ערכים מיוחדים נוספים:

```
a = nothing
if a == nothing
    println("a is nothing")
end
a = 1 / 0
println(a, "   is a > 0 ?  ", a > 0)   # Inf   is a > 0 ?  true
println(-1 / 0)                       # -Inf
println(0 / 0)                        # NaN
```

בהשוואות, `Inf` מתנהג כמו מספר חיובי גדול מאוד, ואילו השוואות עם `NaN` נכשלות, לרבות `== NaN`. לכן בבדיקות נתונים נשתמש ב־`isfinite` במקום להסתפק בהשוואה לאפס.

לבסוף, החזרת ערך והדפסה הן פעולות שונות. פונקציה שמדפיסה שולחת את התשובה למסך במקום להחזיר אותה לקוד שקרא לה:

```
function print_square(x)
    println(x^2)
end
answer = print_square(4)   # prints 16
println(answer)            # nothing
answer = square(4)
println(answer + 1)        # 17
```

אחרי הקריאה הראשונה המשתנה `answer` מכיל `nothing`; אחרי השנייה הוא מכיל מספר שאפשר להוסיף לו. נחזיר את התוצאות מן הפונקציה, ונדפיס אותן במקום הקריאה כאשר נרצה לראותן.

5.5 שינוי מערכים ומוסכמת !

פונקציה יכולה לשנות מערך שהועבר אליה. לפי המוסכמה, בשמה של פונקציה כזאת מופיע הסימן !:

```
function double_values!(values)
    for i in eachindex(values)
        values[i] = 2 * values[i]
    end
    return values
end
values = [1, 2, 3]
another_name = values
double_values!(values)
println(values)          # [2, 4, 6]
println(another_name)    # [2, 4, 6]
```

שני השמות מציגים אותו פלט מפני שהם מפנים לאותו וקטור. הפונקציה `copy` יוצרת וקטור עצמאי:

```
values = [1, 2, 3]
another_name = values
another_vector = copy(values)
double_values!(values)
println(values)          # [2, 4, 6]
println(another_name)    # [2, 4, 6]
println(another_vector)  # [1, 2, 3]
```

השמה מחדש לשם גורמת לו להפנות לעצם אחר, בלי לשנות את העצם המקורי או להשפיע על השמות האחרים שמפנים אליו.

סימן הקריאה הוא הבטחה לקוראים

Julia אינה אוכפת את השימוש ב־`!`; זוהי מוסכמה המודיעה לקוראים שאחד הארגומנטים ישתנה. פונקציה שמשנה וקטור צריכה לציין זאת בשמה. פונקציה שמחשבת וקטור חדש אינה אמורה להשתמש בסימן.

5.6 הפעלה על איברי מערך

את רוב החישובים הסקלריים אפשר להפעיל על כל איברי המערך בבת אחת. לשם כך נוסיף נקודה לפני האופרטור או לפני סוגרי הקריאה לפונקציה:

```
xs = [1, 2, 3]
println(square.(xs))                  # [1, 4, 9]
println(sqrt.([1.0, 4.0, 9.0]))       # [1.0, 2.0, 3.0]
println(shifted.(xs, 10))             # [11, 12, 13]
println(affine.(xs; scale = 2, offset = 1))   # [3, 5, 7]
println(xs .^ 2 .+ 1)                 # [2, 5, 10]
println(xs .* [10, 20, 30])           # [10, 40, 90]
```

הביטוי `shifted.(xs, 10)` קורא לפונקציה בעלת שני הארגומנטים פעם אחת לכל איבר של `xs`, ובכל קריאה משתמש באותו סקלר 10. ארגומנטי המפתח מועברים ללא שינוי לכל קריאה.

המאקרו `@.` מוסיף עבורנו את הנקודות, ולכן אפשר לכתוב נוסחה סקלרית בצורתה הטבעית:

```
grid = [-2.0, -1.0, 0.0, 1.0, 2.0]           # dimensionless coordinates
gaussian_macro = @. exp(-0.5 * grid^2)
println(round.(gaussian_macro; digits = 4))   # [0.1353, 0.6065, 1.0,
0.6065, 0.1353]
```

השמה שנמצאת בתוך `@.` הופכת להשמה לתוך המערך הקיים:

```
gaussian_buffer = zeros(length(grid))
@. gaussian_buffer = exp(-0.5 * grid^2)
```

מערך העבודה נשאר אותו עצם ורק איבריו משתנים. נשאיר פעולות צמצום כמו `sum` וקריאות כמו `println` מחוץ למאקרו, אלא אם נרצה להפעיל גם אותן איבר־איבר.

5.7 הבנות רשימה ומטריצה

הבנת רשימה בונה מערך מתוך לולאה בביטוי אחד. נתחיל בגרסת הלולאה, שתשמש להשוואה:

```
squares_loop = zeros(Int, 4)
for i in 1:4
    squares_loop[i] = i^2
end
squares_comp = [i^2 for i in 1:4]
println(squares_comp)             # [1, 4, 9, 16]
println(squares_loop == squares_comp)   # true
```

פסיק בין ביטויי היצירה בונה מטריצה:

```
table_loop = zeros(Int, 3, 4) # 3 rows, 4 columns
for i in 1:3
    for j in 1:4
        table_loop[i, j] = i * j
    end
end
table_comp = [i * j for i in 1:3, j in 1:4]
println(table_comp)               # [1 2 3 4; 2 4 6 8; 3 6 9 12]
println(table_loop == table_comp) # true
```

כל הבנת רשימה מחזירה מערך חדש. נשתמש בלולאה מפורשת כאשר איטרציה אחת צריכה לבצע כמה פעולות שונות, ובהבנת רשימה כאשר כל איטרציה מייצרת איבר אחד בדיוק.

5.8 מחלקים ומספרים ראשוניים

מחלק חיובי של n מחלק אותו ללא שארית. אם נכתוב תחילה פונקציה למציאת המחלקים, נוכל להגדיר מספר ראשוני בלי להסתמך על קיצור דרך:

```
function divisors(n)
    result = Int[]
    for d in 1:n
        if n % d == 0
            push!(result, d)
        end
    end
    return result
end

println(divisors(1))    # [1]
println(divisors(2))    # [1, 2]
println(divisors(12))   # [1, 2, 3, 4, 6, 12]
println(divisors(0))    # Int64[]
```

הלולאה עוברת עד n מפני שביקשנו למצוא את כל המחלקים. שיטה מהירה יותר בודקת רק עד `isqrt(n)` ומצמידה לכל מחלק d את המחלק n/d; את הגרסה הפשוטה קל יותר לבדוק במבט.

הבנת הרשימה מן הסעיף הקודם בונה אותו וקטור:

```
list_divisors = [d for d in 1:12 if 12 % d == 0]
println(list_divisors == divisors(12))   # true
```

כעת נגדיר: למספר ראשוני יש בדיוק שני מחלקים חיוביים, 1 והמספר עצמו:

```
function is_prime(n)
    return length(divisors(n)) == 2
end

println(is_prime(1))    # false: one divisor
println(is_prime(2))    # true: 1 and 2
println(is_prime(12))   # false
println(is_prime(97))   # true
println([n for n in 1:30 if is_prime(n)])   # [2, 3, 5, 7, 11, 13, 17,
19, 23, 29]
```

לפי ההגדרה הזאת 2 הוא ראשוני ו־1 אינו ראשוני — שתי נקודות שבהן קל לטעות. בדיקת ראשוניות המחפשת מחלק בין 2 ל־$\sqrt{n}$ היא אלגוריתם אחר ושימושי, הנובע מן ההגדרה אך אינו ההגדרה עצמה.

5.9 אינטגרציית אוילר למטוטלת

מסה נקודתית תלויה על מוט קשיח באורך L, המחובר לציר ללא חיכוך. הזווית θ נמדדת מן האנך כלפי מטה, וחיובית נגד כיוון השעון; $\omega = \dot\theta$ היא המהירות הזוויתית. מומנט הכוח המחזיר הוא $-mgL\sin\theta$, וחלוקתו במומנט ההתמד mL^2 נותנת את משוואת התנועה

$$\ddot\theta = -\frac{g}{L}\sin\theta.$$

נשמור על פונקציית הסינוס המלאה, משום שאנו רוצים לחקור את התנועה מעבר לקירוב הזוויות הקטנות $\sin\theta \approx \theta$.

```
function pendulum_rhs(state; length = 1.0, gravity = 9.81)
    θ, ω = state
    return ω[, -(gravity / length) * sinθ()]
end
```

המצב הוא הזוג (θ, ω), והפונקציה מחזירה את שתי נגזרותיו לפי הזמן. שיטת אוילר קדימה מקדמת את המצב בעזרת הנגזרת בזמן הנוכחי:

$$\text{state}_{\text{next}} = \text{state}_{\text{now}} + \Delta t \cdot \text{derivative}_{\text{now}}.$$

בזכות הסימטריה, חציית האפס הראשונה בכיוון הירידה מתרחשת כעבור רבע תנודה מלאה, ולכן זמן המחזור הוא פי ארבעה מן הזמן הזה:

```
function pendulum_periodθ(0; length = 1.0, gravity = 9.81, dt = 0.001)
    state = θ[0, 0.0]
    t = 0.0
    while true
        derivative = pendulum_rhs(state; length = length, gravity =
gravity)
        θ = state[1]
        θ_next = θ + dt * derivative[1]
        state[1] = θ_next
        state[2] = state[2] + dt * derivative[2]
        if θ_next <= 0
            crossing = t + dt * θ / θ( - θ_next)
            return 4 * crossing
        end
        t += dt
    end
end
```

את זמן החצייה נמצא באינטרפולציה בין הדגימה החיובית האחרונה לדגימה השלילית הראשונה, כדי שהתשובה לא תקפוץ בכפולות של צעד הזמן. הלולאה מסתיימת בחצייה הראשונה כלפי מטה; מטוטלת שמשוחררת ממנוחה בזווית θ_0 נעה בין $-\theta_0$ ל־θ_0.

תיבת פיסיקה: מטוטלת על מוט קשיח

המודל מתאר מסה נקודתית על מוט קשיח חסר מסה באורך L במטרים, ציר קבוע ללא חיכוך ושדה כבידה אחיד g ביחידות $\mathrm{m/s^2}$, ללא ריסון וללא כוח מניע חיצוני. מוט קשיח מאפשר שחרור גם מעל 90°; חוט גמיש המשוחרר ממנוחה בזווית כזאת היה מתרפה. הזוויות נמדדות ברדיאנים מן האנך כלפי מטה, וחיוביות נגד כיוון השעון; ω נמדדת ברדיאנים לשנייה.

$$\ddot{\theta} = -\frac{g}{L}\sin\theta, \qquad \frac{\mathrm{d}}{\mathrm{d}t}\begin{pmatrix}\theta\\ \omega\end{pmatrix} = \begin{pmatrix}\omega\\ -(g/L)\sin\theta\end{pmatrix}.$$

גבול הזוויות הקטנות $\sin\theta \approx \theta$ נותן $T_0 = 2\pi\sqrt{L/g}$. זמן המחזור הלא־ליניארי גדל עם המשרעת ומתבדר כאשר $\theta_0 \to \pi$. אינטגרציית אוילר משנה בהדרגה את אנרגיית המתנד, ולכן זמן המחזור המתקבל הוא קירוב לצורכי לימוד ואינו מדידה מדויקת. המודל אינו כולל מגע בקרקע, גמישות של המוט או תנועה של הציר.

5.10 המחזור כתלות בזווית השחרור

כעת, משהמחזור מחושב באמצעות פונקציה, נוכל לחשב אותו לכל זווית שחרור בעזרת הבנת רשימה:

```
using Plots
let
    angles_deg = 5:5:175
    periods = [pendulum_period(deg2radθ()) for θ in angles_deg]
    p = plot(angles_deg, periods; xlabel = "Release angle (degrees)",
        ylabel = "Period (s)", title = "Pendulum period", legend = false,
        linewidth = 2, marker = :circle, markersize = 3)
    savefig(p, joinpath(@__DIR__, "week05-pendulum-period.png"))
    display(p)
end
```

העקומה שטוחה בקרבת האפס ועולה בתלילות כאשר זווית השחרור מתקרבת ל־180°. עבור $L = 1$ מטר, זמן המחזור בזוויות קטנות הוא כ־2.006 שניות, וב־90° הוא כבר ארוך בכ־18%. הגרף מתקבל מן המודל ואינו מדידה של מוט מסוים.

5.11 מבט קדימה

בתרגול המתאים נבנה פונקציה קטנה אחת לפני שנתחיל את שיעורי הבית. מטלת החובה הנפרדת מבקשת לכתוב את אגף ימין של משוואות תנועת קליע עם גרר ריבועי, לבצע אינטגרציית אוילר ולחפש בין זוויות השיגור את הזווית הנותנת את הטווח הגדול ביותר. בבונוס הרשות נבצע אינטגרציה למטוטלת כפולה, ששתי משוואותיה המצומדות מופיעות כמשוואה וקטורית אחת. חומרי הבונוס וההעמקה אינם דרישת קדם להמשך הלימודים.

ניקח איתנו את ההרגל הבא: כשחישוב מופיע בפעם השנייה, ניתן לו שם, נבהיר מה הוא מקבל ומה הוא מחזיר, ונשתמש בו בכל מקום שנזדקק לו.